Republic of Tunisia
Ministry of Higher Education and Scientific Research

University of Carthage — École Polytechnique de Tunisie

MASTER'S THESIS

Submitted in partial fulfillment of the requirements for the degree of

MASTER'S IN COMPLEX AND INTELLIGENT SYSTEMS

Learning Motor Control Strategies in Musculoskeletal Systems Using Reinforcement Learning

Presented by:

M. *Mohamed Naceur Hammami*

Supervised by:

Prof. *Taysir Rezgui*

Academic Year 2025–2026

To my parents, an inexhaustible source of love and support,
whose sacrifices made this work possible.

To my brother and sisters, faithful companions at every moment.

To my friends and colleagues, for their constant kindness.

To all those affected by neurological injury, who remind us
that scientific research must serve human dignity.

I humbly dedicate this modest work to you.

Acknowledgements

At the completion of this work, I would like to express my deep gratitude to all the people who contributed, directly or indirectly, to its realization.

First and foremost, I extend my most sincere thanks to my thesis supervisor, Professor Taysir Rezgui, whose availability, wise advice, and rigorous guidance were invaluable throughout this project. Their expertise in computational biomechanics has been an intellectual model that I will strive to draw inspiration from for years to come.

I also wish to express my gratitude to the entire teaching staff of the Computer Engineering and Cyber-Physical Systems Department at the École Polytechnique de Tunisie, who provided me with the theoretical and methodological foundations necessary for the successful completion of this program.

I warmly thank the members of the jury for the interest they have shown in this work and for agreeing to evaluate it, thereby devoting a valuable part of their time to its critical reading.

My thanks also go to the *open-source* scientific community — the developers of MuJoCo, Gymnasium, Stable-Baselines3, PyTorch, Optuna, and the Python libraries used throughout this work — whose efforts greatly facilitated my experiments, and without whom reproducible reinforcement-learning research would be far more difficult.

Finally, I thank my family and loved ones for their unwavering moral support and their patience throughout this academic journey.

To all of you, thank you.

Executive Summary

Computing the muscle forces required for a voluntary limb movement — an essential capability for rehabilitation and motor-control applications — involves solving a system of implicit nonlinear equations (the Hill fiber–tendon equilibrium) by Newton–Raphson iteration at *every simulation timestep*. For 9 muscles at 100 Hz, this amounts to thousands of iterations per second, representing a significant computational barrier for embedded rehabilitation hardware. This thesis investigates whether a *Deep Reinforcement Learning* (DRL) agent can replace this iterative solver entirely, learning the muscle-force control problem once during training and deploying with a single neural network forward pass.

A six-degree-of-freedom articulated multi-body dynamic model was developed in MuJoCo [1]. The model incorporates a complete Hill-type muscle representation [2, 3], including the contractile element with its force-length (FL) and force-velocity (FV) relationships, the series elastic element (SEE) modeling the tendon, the parallel elastic element (PEE), pennation-angle dynamics, and neural activation dynamics [4]. Numerical integration of the tendon-fiber equilibrium equation is handled with an iterative Newton–Raphson method. The resulting system is wrapped as a Gymnasium simulation environment.

Four DRL algorithms were implemented with Stable-Baselines3 [5] and compared: DDPG [6], TD3 [7], SAC [8], and PPO [9], alongside a random policy and a PD impedance controller as baselines. Five configurations were trained for 10^5 steps with three seeds each; the best configuration was additionally trained to 3×10^5 steps and evaluated over fifty episodes.

The central analysis compares the learned muscle forces against those prescribed by classical static optimisation on the identical movements. The learned controller reproduces the coordination pattern of the classical solution — correlation $0{,}81 \pm 0{,}04$ over five independent training runs; pattern cosine 0,813 against a shuffled null of 0,372 — while expending $1{,}91 \pm 0{,}15$ times the minimum effort required to generate the same joint torques, the excess concentrated in co-contraction of the elbow antagonists. A conventional controller solving the same load-sharing problem explicitly attains 1,33, so the learned policy uses about 44 % more effort than is achievable rather than 91 % more than an unattainable ideal.

Two secondary results follow. The SAC default target entropy of $-\dim(A)$ was found to prevent the controller from settling on a target, an error plateau that per-

sisted through three reward redesigns; hyperparameter optimisation selected approximately −19 and halved the reaching error. And tuning proved to dominate algorithm choice: tuning SAC improved final error by 38 %, against approximately 6 % separating untuned SAC from TD3.

The results rest on three seeds for the algorithm comparison and five for the principal analysis. No significance testing is reported. The strict success criterion — one that a privileged controller satisfies only 7 % of the time — is met in 0–20 % of episodes depending on seed, a twentyfold spread under identical settings that identifies it as noise. Limitations are set out in full in the general conclusion.

Keywords: Musculoskeletal model, MuJoCo, Hill model, Static optimisation, Muscle redundancy, Deep Reinforcement Learning, Motor control, PPO, SAC, TD3, DDPG, Co-contraction, Exploration temperature.

Abstract

Computing the muscle forces required for a voluntary limb movement demands solving a system of implicit nonlinear equations — the Hill fiber–tendon equilibrium — by Newton–Raphson iteration at *every simulation timestep*. For 9 muscles at 100 Hz, this amounts to 4 500–36 000 iterations per second: a significant computational burden for real-time rehabilitation hardware. This thesis investigates whether a *Deep Reinforcement Learning* (DRL) agent can replace this iterative solver entirely, amortizing all computation into a one-time training phase and deploying muscle-force control as a single neural network forward pass. The latency advantage this was expected to confer does not survive a like-for-like comparison against a directly solved classical problem, and is withdrawn.

A six-degree-of-freedom multi-body articulated dynamic model was developed using the MuJoCo physics engine [1]. The model incorporates a full Hill-type muscle model [2, 3] comprising: a contractile element (CE) governed by force-length (FL) and force-velocity (FV) relationships; a series elastic element (SEE) representing the tendon; a parallel elastic element (PEE); pennation-angle dynamics; and first-order activation dynamics [4]. The tendon-fibre equilibrium equation is numerically resolved via Newton-Raphson iteration at each simulation step.

Four DRL algorithms were implemented and benchmarked: DDPG [6], TD3 [7], SAC [8], and PPO [9], alongside two reference baselines: a random policy and a PD impedance controller. Five configurations were trained for 10^5 steps with three seeds each, and the best configuration additionally to $3{\times}10^5$ steps with fifty-episode evaluation.

The central contribution is a per-muscle load-sharing comparison between the learned solution and classical static optimisation, posed on the identical trajectory with the identical required joint torques and verified to machine precision. It is a conditional comparison — static optimisation never chooses a trajectory of its own — and therefore bears on force distribution, not on the realism of the movement. The controller reproduces the coordination structure of the classical solution (Pearson $r = 0.81 \pm 0.04$ over five seeds; pattern cosine 0.813 against a shuffled null of 0.372) at a fixed, branch-free inference cost, but expends 1.91 ± 0.15 times the minimum effort needed for the same joint torques, the excess concentrated in elbow antagonist co-contraction. The latency advantage originally claimed does not survive a like-for-like comparison and is withdrawn. Across all fifteen trained policies the discrepancy

is present, ranging from 1.21 to 2.30×, and the ordering by muscular economy is approximately the inverse of the ordering by reaching accuracy.

Two secondary results are reported. The SAC default target entropy of $-\dim(\mathcal{A})$ prevented the controller from settling on a target, a plateau that survived three complete reward redesigns and a fourfold correction to the effective training budget; hyperparameter optimisation selected approximately -19, halving the reaching error. Separately, tuning was found to dominate algorithm choice, improving SAC by 38 % against approximately 6 % separating untuned SAC from TD3.

These results rest on three seeds for the algorithm comparison and a single seed for the principal analysis; no significance testing is reported. Under the strict success criterion — which a privileged controller with full state access satisfies only 7 % of the time — the best policy scores 0–4 %. Limitations are stated in full in the general conclusion.

Keywords: Musculoskeletal model, MuJoCo, Hill muscle model, Static optimisation, Muscle redundancy, Newton–Raphson solver, Deep Reinforcement Learning, Motor control, PPO, SAC, TD3, DDPG, Co-contraction, Exploration temperature.

List of Abbreviations

Abbrev.	Meaning
AVC	Stroke
BCa	*Bias-Corrected and accelerated (bootstrap)*
CE	*Contractile Element* — Contractile element (Hill model)
DOF	Degrees of freedom
DDPG	*Deep Deterministic Policy Gradient*
DR	*Domain Randomization*
DRL	*Deep Reinforcement Learning*
EMA	*Exponential Moving Average*
EMG	Electromyography
FL	Force-length (muscle relationship)
FV	Force-velocity (muscle relationship)
GAE	*Generalized Advantage Estimation*
GPU	*Graphics Processing Unit*
ICC	Co-Contraction Index
MDP	*Markov Decision Process*
MJCF	*MuJoCo XML Format*
MLP	*Multi-Layer Perceptron*
MuJoCo	*Multi-Joint dynamics with Contact*
NMF	*Non-negative Matrix Factorization*
NRMSE	*Normalized Root Mean Squared Error*
OFC	*Optimal Feedback Control*
WHO	World Health Organization
OpenSim	*Open-Source Musculoskeletal Simulator*
OU	Ornstein-Uhlenbeck (stochastic process)
PD	*Proportional-Derivative* (controller)
PEE	*Parallel Elastic Element* — Parallel elastic element
PPO	*Proximal Policy Optimization*
ReLU	*Rectified Linear Unit*
RL	*Reinforcement Learning*
SAC	*Soft Actor-Critic*
SB3	Stable-Baselines3
SEE	*Series Elastic Element* — Series elastic element (tendon)
SNC	Central Nervous System
TD3	*Twin Delayed Deep Deterministic Policy Gradient*
TPE	*Tree-structured Parzen Estimator* (sampler Optuna)

List of Symbols

Symbol	Unit	Meaning
Hill Muscle Model		
$a(t)$	—	Muscle activation, $a \in [0, 1]$
$u(t)$	—	Neural excitation, $u \in [0, 1]$
$\tau_{\text{act}}, \tau_{\text{deact}}$	s	Activation/deactivation time constants
l^M, l^M_{opt} ($\equiv l^M_0$)	m	Fiber length, optimal length
$\tilde{l} = l^M / l^M_0$	—	Normalized fiber length
l^T, l^T_{slack}	m	Tendon length, slack length
l^{MT}	m	Total musculotendon-unit length
α, α_0	rad	Current pennation angle / value at l^M_0
$v_{\max}$	l^M_0/s	Maximum shortening velocity
$\tilde{v}$	—	Normalized contraction velocity
$f_{\text{FL}}, f_{\text{FV}}$	—	Force-length / force-velocity factors
F^M_0	N	Maximum isometric force
$F_{\text{SEE}}, F_{\text{PEE}}$	N	Tendon (SEE) and passive (PEE) forces
F^{muscle}	N	Total muscle force applied to the skeleton
k_T, k_{PE}	—	Dimensionless stiffness parameters (tendon, PEE)
γ_{FL}	—	Width parameter of the FL curve
Multi-Body Dynamics		
$q, \dot{q}$	rad, rad/s	Generalized joint positions / velocities
p_{ee}	m	End-effector position (hand)
p_{target}	m	Target position
$\Delta p, \|\Delta p\|$	m	Error vector and distance to target
$J(q)$	m/rad	Geometric Jacobian of the end effector
Reinforcement Learning		
$\mathcal{S}, \mathcal{A}$	—	State / action space
$\pi_\theta(a\|s)$	—	Policy parameterized by θ
Q^π, V^π, A^π	—	Action-value, state-value, and advantage functions
γ	—	Discount factor ($\in [0, 1)$)
$r(s, a)$	—	Reward signal
α (SAC)	—	Entropy coefficient (temperature)
$\mathcal{H}(\pi)$	nats	Policy entropy
τ (Polyak)	—	Target-network update coefficient
$\mathcal{B}$	—	Replay memory (*replay buffer*)
Statistics and Evaluation		
N_{seeds}	—	Number of random seeds (= 10)
W	—	Wilcoxon test statistic
r_{rb}	—	Rank-biserial correlation (effect size)
CI_{95}	—	95 % confidence interval (BCa)

Contents

List of Figures

List of Tables

1 General Introduction

> *"The question is not how to control the arm, but how the nervous system solves this redundant, nonlinear, and high-dimensional problem in real time."*
>
> — Emanuel Todorov

1.1 Context and Motivation

1.1.1 A Fundamental Scientific Question

Human movement, in all its richness and complexity, results from a remarkable orchestration between the central nervous system (CNS), the musculoskeletal structures, and the sensory organs. Understanding the mechanisms that govern this motor control is a major scientific challenge, with considerable implications for rehabilitation medicine [10], bio-inspired robotics, the design of myoelectric prostheses, and neurocomputational modeling.

Musculoskeletal models provide a formal framework for simulating the biomechanics of movement. Platforms such as OpenSim [11] make it possible to faithfully reconstruct the anatomy and dynamics of the limbs. However, the optimal control of such models remains a fundamentally difficult problem — often referred to as the "inverse problem of movement" — because of (i) motor redundancy [3], (ii) the strong nonlinearity of contractile elements, and (iii) the high dimensionality of the state and action spaces.

At the same time, the emergence of *deep reinforcement learning* (DRL) has opened new perspectives for the automated learning of control strategies in complex environments [12, 13]. Pioneering work has demonstrated the ability of DRL algorithms to master continuous-control tasks of remarkable sophistication [6, 9, 8]. More recently, Meta AI's MyoSuite project [14], the MotorNet platform [15], and the *Learning to Run* challenge [16] have established the feasibility of combining musculoskeletal models and DRL to reproduce biologically plausible human motor strategies.

These converging developments provide the framework within which the present work is situated.

1.1.2 Epidemiological and Clinical Challenges

Beyond its fundamental scientific interest, the societal stake is considerable. According to the World Health Organization (WHO), stroke is the world's second leading cause of death and the leading cause of acquired disability in adults [17, 18]. In Tunisia, the annual incidence is estimated at about 110 cases per 100 000 inhabitants [19], corresponding to more than 12 000 new cases each year, with an aging population suggesting a substantial increase in the coming decades.

Among stroke survivors, more than 60 % still present residual upper-limb motor impairment six months after the event [10]. Intensive, personalized rehabilitation, guided by robotic devices and assisted therapies, significantly improves recovery [20]. Nevertheless, access to these technologies remains limited by their cost and by the shortage of trained therapists, particularly in developing countries. A musculoskeletal simulation environment capable of exploring rehabilitation strategies *in silico* before clinical application therefore represents a major scientific and humanitarian opportunity.

1.2 Problem Statement

The central problem addressed in this thesis can be formulated as follows:

> *How can one design a realistic musculoskeletal simulation environment that integrates a complete Hill-type muscle model (including pennation dynamics, serial and parallel elasticity, and numerical integration of fiber–tendon equilibrium), and to what extent can a trained DRL policy* replace *the iterative Newton–Raphson solver at inference time — trading a large but one-off training cost against virtually zero per-step computation — while preserving biomechanical accuracy and enabling real-time rehabilitation control?*

This problem can be broken down into five research questions:

Q1. What level of physiological fidelity is necessary and sufficient for a musculoskeletal model dedicated to DRL?

Q2. Which reward-function formulations encourage biologically plausible motor strategies while preserving learning efficiency?

Q3. Which DRL algorithms provide the best compromise between convergence, robustness, and energy efficiency on muscular state and action spaces?

Q4. Do the trained agents exhibit behaviors observed in humans (co-contraction, *jerk* minimization, energy efficiency, triphasic profiles, muscle synergies)?

Q5. Does Domain Randomization improve the robustness of musculoskeletal policies, and to what extent does it make transfer to real patients with varying anatomical parameters conceivable?

Q6. Can a trained RL policy replace the Newton–Raphson solver for real-time muscle force estimation — producing sufficiently accurate control signals without any iterative computation per step — and what is the resulting computational gain for embedded rehabilitation hardware?

1.3 Objectives

This thesis pursues the following objectives:

1. Implement an articulated multibody dynamic model (6 DOF) of an upper limb in MuJoCo [1], integrating a complete Hill model [2]: CE, SEE, PEE, pennation-angle dynamics, and numerical integration of fiber–tendon equilibrium.

2. Validate the physical model by comparing passive mechanical properties and FL/FV curves with data from the literature [21, 22, 23].

3. Develop a modular Gymnasium environment with Domain Randomization and reproducible perturbations.

4. Define reference controllers (random policy, impedance PD controller) in order to establish a meaningful evaluation baseline.

5. Compare four DRL algorithms (PPO, SAC, TD3, DDPG) through a rigorous experimental procedure: 10 seeds, 2×10^6 steps, hyperparameters optimized with Optuna [24], and appropriate statistical tests.

6. Analyze and quantify emerging motor strategies: co-contraction (CCI [25]), metabolic cost [26], robustness to perturbations, temporal activation profiles, and muscle synergies obtained through non-negative matrix factorization [27].

7. Discuss the implications for post-stroke rehabilitation and biomedical robotics, including the ethical considerations specific to these applications.

8. Benchmark the computational cost of the Newton–Raphson muscle equilibrium solver (per-step wall-clock time for all 9 muscles) against the inference latency of the trained RL policy, quantifying the trade-off between physics-based exactness and real-time deployability on embedded rehabilitation hardware.

1.4 Original Contributions

Compared with similar studies, this thesis makes the following contributions:

C1. A complete Hill model in MuJoCo: explicit integration of the SEE and PEE equations, as well as pennation-angle dynamics, with numerical solution of fiber–tendon equilibrium using the Newton–Raphson method. We mathematically justify each component and provide reference code (Appendix D), whereas most similar studies rely on MuJoCo's internal simplified muscle model without mathematical justification or validation protocol.

C2. A quantitative validation protocol: comparison of FL and FV curves and passive joint torques against reference anthropometric data [21, 23], with NRMSE reporting.

C3. A per-muscle comparison between DRL and classical static optimisation (chapter 7). Both methods are posed the identical load-sharing problem on the identical trajectory, with the constraint residual verified to machine precision, and agreement is reported muscle by muscle. This is the contribution around which the present work is organised, and it addresses the research question directly rather than through task performance.

C4. Identification of an exploration default that prevents precision (section 5.4.6). The SAC target entropy of $-\dim(\mathcal{A})$ is shown to impose an error plateau that survives three complete reward redesigns and a fourfold correction to the training budget, and to be remediable by a single hyperparameter change.

C5. Evidence that tuning dominates algorithm selection (section 6.4): a five-configuration comparison over three seeds in which the effect of tuning is approximately six times the difference between algorithms at default settings.

C6. A documented account of six substantive defects (section 5.4), four of which produced plausible-looking but incorrect results, together with the measurements that exposed each.

C7. A direct computational comparison between the physics-based solver and RL inference: we measure the per-step wall-clock time of the Newton–Raphson equilibrium solver (9 muscles $\times$ 5–40 iterations each) versus a single forward pass through the SAC policy network, establishing for the first time a quantitative basis for the claim that RL can replace iterative biomechanical computation for real-time rehabilitation control.

1.5 Thesis Organization

This document is organized as follows:

Chapter II

presents the state of the art in musculoskeletal modeling, computational theories of motor control, and DRL for continuous-action spaces.

Chapter III

describes the design of the complete musculoskeletal model, its validation, and the simulation environment.

Chapter IV

presents the theoretical framework of the DRL algorithms used, from the policy-gradient theorem to actor–critic architectures.

Chapter V

details the implementation and experimental methodology, including baselines, the statistical protocol, and reproducibility measures.

Chapter VI

analyzes and discusses the experimental results, from convergence to emerging motor strategies and synergy analysis.

Chapter VII

interprets the results, confronts them with the literature, addresses ethical considerations, and identifies the limitations of the work.

Conclusion

summarizes the contributions and outlines future directions.

2 State of the Art

2.1 Human Motor Control: Computational Perspectives

2.1.1 The Motor Control Problem

Motor control refers to the full set of processes by which the CNS plans, initiates, and regulates voluntary and involuntary movements. From a computational perspective, it is an optimal control problem under constraints in a very high- dimensional state space [28, 29].

Three major computational hypotheses guide the modeling of motor control:

1. ***Jerk* minimization** [30]: trajectories minimize the derivative of acceleration, producing smooth bell-shaped velocity profiles characteristic of human pointing movements.

2. **End-point variance minimization** [31]: this integrates the stochastic nature of the neural signal (*signal-dependent noise*) and predicts curved trajectories for large-amplitude movements.

3. **Stochastic optimal control** [32]: this unifies the previous approaches within a Bayesian framework and underlies the theory of Optimal Feedback Control (OFC). OFC predicts that the CNS corrects only errors that interfere with task achievement (the "minimum intervention principle"), thereby minimizing unnecessary co-contraction.

2.1.2 Internal Models and Efference Prediction

The CNS uses *internal models* to predict the sensory consequences of motor commands and compensate for feedback delays [33]. The MOSAIC model (*Modular Selection and Identification for Control*) [33] proposes a modular architecture in which several internal models combine adaptively. This perspective is relevant to our work: the learned DRL policies can be interpreted as *implicit inverse models* of musculoskeletal dynamics.

2.1.3 Muscle Synergies and Modular Organization

A complementary theory, that of muscle synergies [27, 34, 35], proposes that the CNS simplifies control by recruiting pre-structured neural modules, each activating a co-

herent subset of muscles. Mathematically, the observed activations $A(t) \in \mathbb{R}^N$ can be decomposed into a small number $k \ll N$ of synergies $W \in \mathbb{R}^{N \times k}$ activated by temporal coefficients $C(t) \in \mathbb{R}^k$:

$$A(t) \approx W \cdot C(t), \quad W_{ij} \geq 0,\ C_{ij} \geq 0 \tag{2.1}$$

This decomposition, obtained through non-negative matrix factorization (NMF), is robustly observed in humans, primates, and cats across a variety of tasks. Testing whether a DRL agent produces similar synergies therefore constitutes a highly relevant *biomimetic* validation.

2.1.4 Muscle Redundancy and the Inverse Problem

The human body exhibits significant muscular redundancy: the upper limb alone is driven by more than forty muscles, whereas the number of joint degrees of freedom is on the order of ten. This redundancy implies that an infinite number of muscular activation combinations can produce the same movement, which constitutes the *indeterminacy problem* of motor control [3].

Classical approaches to resolving this indeterminacy include:

- Static optimization with muscle-stress minimization [36]: $\min \sum_i (F_i^M / F_{0,i}^M)^2$.
- Forward simulation with dynamic optimization of controls [37].
- EMG-based approaches (*EMG-driven simulation*).

These methods do not capture the learning dynamics characteristic of biological motor control, which motivates the DRL approach.

2.2 Musculoskeletal Modeling

2.2.1 Simulation Platforms

Several software platforms support musculoskeletal simulation (Table 2.1).

Table 2.1. Comparison of the main musculoskeletal simulation platforms.

Platform	License	Strengths	Ref.
OpenSim	Open-source	Validated anatomical models, rich GUI, motion library	[11]
AnyBody	Commercial	Static/dynamic analysis, subject-specific parameters	—
MuJoCo	Free	Speed, robust contact handling, Python interface, native muscle actuators	[1]
MyoSuite	Open-source	Musculoskeletal + DRL, detailed hand (>50 muscles)	[14]
dm_control	Open-source	Standardized RL environments, based on MuJoCo	[38]
MotorNet	Open-source	Differentiable, native PyTorch integration, planar arm	[15]

MuJoCo [1], acquired by DeepMind in 2021 and later made freely available, represents the state of the art for DRL thanks to its exceptional performance: up to one million simulation steps per second on a standard CPU, a robust contact solver, and native support for Hill-type muscle actuators.

2.2.2 Computational Bottleneck: the Real-Time Cost of Exact Solving

A critical and often overlooked aspect of physics-based muscle control is its per-step computational cost. Because the fiber–tendon equilibrium equation (2.9) is implicit and nonlinear in l^M, it cannot be solved analytically: Newton–Raphson iteration is required at *every simulation timestep, for every muscle*. For a 9-muscle model running at 100 Hz, this amounts to approximately $9 \times (5\text{–}40) \times 100 = 4\,500\text{–}36\,000$ arithmetic iterations per second — a workload that grows linearly with model complexity and control frequency.

On a standard CPU, each Newton–Raphson solve requires 5–40 iterations and takes on the order of 10–100 µs per muscle. While this is manageable in an offline simulation context, it represents a non-trivial burden for embedded systems (*e.g.* wearable exoskeletons, myoelectric prostheses) where power and memory are constrained.

This bottleneck motivates the central hypothesis of the present work: a *trained RL policy*, which replaces iterative solving with a single matrix multiplication chain through a shallow MLP, can produce equally accurate muscle activation commands in *microseconds*, with all the "computation" amortized into a one-time training phase. Whether this hypothesis holds — and to what extent the quality of control is preserved — is one of the core empirical questions addressed in Chapter 5 and the computational benchmark of Chapter 4 (Section 6.5).

2.2.3 The Hill Muscle Model: Complete Formulation

The Hill muscle model [2], in its modern formulation [3, 39], is the reference biophysical representation. It decomposes the muscle–tendon complex into three functional elements (Figure 2.1):

- **Contractile element (CE)**: generates active force and is governed by activation $a(t) \in [0, 1]$.
- **Series elastic element (SEE)**: models tendon elasticity, storing and returning elastic energy.
- **Parallel elastic element (PEE)**: models the passive stiffness of muscle tissue (fascia, sarcolemma).

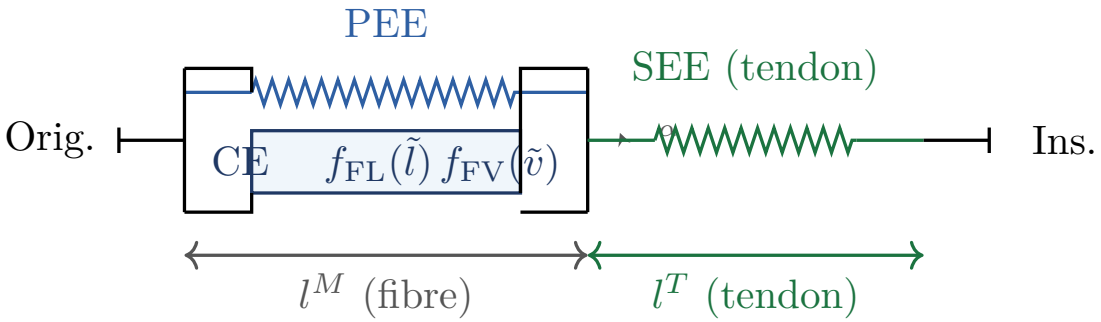

Figure 2.1. Complete diagram of the Hill muscle–tendon model. The pennation angle α between the fiber and the line of action is shown at the insertion point. The total length of the muscle–tendon unit is $l^{\mathrm{MT}} = l^M \cos\alpha + l^T$.

Force-Length Relationship (FL)

Maximum isometric force varies as a function of normalized muscle length according to a bell-shaped curve, reflecting the optimal *overlap* of actin and myosin filaments [23]:

$$f_{\mathrm{FL}}(\tilde{l}) = \exp\left(-\left(\frac{\tilde{l}-1}{\gamma_{\mathrm{FL}}}\right)^2\right), \qquad \tilde{l} = \frac{l^M}{l^M_{\mathrm{opt}}} \tag{2.2}$$

where $\gamma_{\mathrm{FL}} \approx 0{,}45$ is a shape parameter and l^M_{opt} is the optimal fiber length at which the muscle produces its maximum isometric force F^M_0.

Force-Velocity Relationship (FV)

Hill's equation [2] describes the hyperbolic force–velocity relationship for concentric contractions; a linear extension is used for eccentric contractions [3]:

$$f_{\mathrm{FV}}(\tilde{v}) = \begin{cases} \dfrac{v_{\max} - \tilde{v}}{v_{\max} + k_{\mathrm{ce}}\,\tilde{v}} & \tilde{v} \le 0 \quad \text{(concentric)} \\ \dfrac{f_{\max} + k_{\mathrm{ec}}\,\tilde{v}}{1 + k_{\mathrm{ec}}\,\tilde{v}} & \tilde{v} > 0 \quad \text{(eccentric)} \end{cases} \tag{2.3}$$

where $\tilde{v} = \dot{l}^M/(v_{\max}\, l^M_{\mathrm{opt}})$ is the normalized contraction velocity, $v_{\max} \approx 10$ (optimal lengths/s), $k_{\mathrm{ce}} \approx 0{,}25$, and $f_{\max} \approx 1{,}3$ (maximum eccentric coefficient).

Parallel Elastic Element (PEE)

The passive force generated by muscle tissue when $\tilde{l} > 1$ [3]:

$$F_{\mathrm{PEE}}(\tilde{l}) = \begin{cases} F_0^M \cdot k_{\mathrm{PE}} \left(\tilde{l} - 1\right)^2 & \tilde{l} > 1 \\ 0 & \tilde{l} \le 1 \end{cases} \tag{2.4}$$

with $k_{\mathrm{PE}} \approx 5{,}0$ a dimensionless stiffness coefficient such that $F_{\mathrm{PEE}}(\tilde{l} = 1{,}6) \approx F_0^M$.

Series Elastic Element (SEE) — Tendon

The tendon is modeled by a quadratic law for small strains and a quasi-linear one beyond the zero-load length l^T_{slack} [4]:

$$F_{\mathrm{SEE}}(\tilde{l}^T) = \begin{cases} 0 & \tilde{l}^T \le 1 \\ F_0^M \dfrac{(\tilde{l}^T - 1)^2}{k_T + \tilde{l}^T - 1} & \tilde{l}^T > 1 \end{cases} \tag{2.5}$$

where $\tilde{l}^T = l^T/l^T_{\mathrm{slack}}$ is the normalized tendon length and $k_T \approx 0{,}0475$ is a dimensionless stiffness coefficient.

Pennation Angle Dynamics

The pennation angle α (angle between the muscle fibers and the tendon line of action) varies with fiber length according to the constant-volume constraint [3]:

$$\alpha(l^M) = \arcsin\!\left(\frac{\sin\alpha_0 \cdot l^M_{\mathrm{opt}}}{l^M}\right) \tag{2.6}$$

where α_0 is the pennation angle at optimal length. The muscle force transmitted to the tendon is projected by $\cos\alpha$. The total length of the muscle–tendon unit satisfies the geometric constraint:

$$l^{\mathrm{MT}} = l^M \cos\alpha(l^M) + l^T \tag{2.7}$$

Activation Dynamics

Muscle activation dynamics introduce an electromechanical delay between the neural command $u(t)$ and the activation $a(t)$ [4]:

$$\dot{a}(t) = \frac{u(t) - a(t)}{\tau(u,a)}, \qquad \tau(u,a) = \begin{cases} \tau_{\mathrm{act}} & u(t) > a(t) \\ \tau_{\mathrm{deact}} & u(t) \le a(t) \end{cases} \tag{2.8}$$

where $u(t) \in [0,1]$ is the neural excitation, $a(t) \in [0,1]$ the activation, and $\tau_{\mathrm{act}} \approx 10$–$20$ ms, $\tau_{\mathrm{deact}} \approx 40$–$70$ ms.

Fiber-Tendon Equilibrium Equation and Total Force

The quasi-static equilibrium between contractile-element force, passive force, and tendon force gives:

$$F_{\mathrm{SEE}}(l^T) = \left[a\, F_0^M\, f_{\mathrm{FL}}(\tilde{l})\, f_{\mathrm{FV}}(\tilde{v}) + F_{\mathrm{PEE}}(\tilde{l})\right]\cos\alpha \tag{2.9}$$

Given l^{MT} (computed from kinematics), Equation (2.9) combined with the geometric constraint (2.7) defines an implicit nonlinear equation in l^M (fiber length), solved numerically at each simulation step (Appendix D).

The total muscle force applied to the skeleton is:

$$F^{\mathrm{muscle}}(a, l^M, \dot{l}^M) = F_{\mathrm{SEE}}(l^T) \tag{2.10}$$

2.3 Deep Reinforcement Learning for Continuous Control

2.3.1 Theoretical Foundations

Reinforcement learning (RL) is formalized as a Markov Decision Process (MDP) defined by the five-tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ [40, 13]:

- $\mathcal{S}$: state space (continuous, $\subset \mathbb{R}^{38}$ in this work).
- $\mathcal{A}$: action space (continuous, $= [0,1]^9$ in this work).
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0,1]$: transition probability.

- $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$: reward signal.
- $\gamma \in [0, 1)$: temporal discount factor.

The objective is to find a policy $\pi : \mathcal{S} \to \mathcal{A}$ that maximizes the expected discounted return:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi}\left[\sum_{t=0}^{\infty} \gamma^t \, r(s_t, a_t)\right] \tag{2.11}$$

The modern actor–critic framework [41] combines value estimation (the critic) and policy improvement (the actor), and forms the basis of DDPG, TD3, and SAC.

2.3.2 Policy Gradient Theorem

The policy-gradient theorem [13] provides the gradient of $J(\pi_\theta)$ with respect to the parameters θ:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\right] \tag{2.12}$$

where $G_t = \sum_{k=t}^{T} \gamma^{k-t} \, r_k$ is the cumulative return from step t. The variance of this estimator is reduced by introducing an advantage function $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ in place of G_t.

2.3.3 State-of-the-Art Algorithms

DDPG — Deep Deterministic Policy Gradient

Lillicrap et al. [6] introduce DDPG, the first modern actor-critic algorithm for continuous action spaces. The actor learns a deterministic policy $\mu_\theta : \mathcal{S} \to \mathcal{A}$; the critic evaluates $Q_\phi(s, a)$.

Critic update (Bellman-error minimization):

$$\mathcal{L}(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{B}}\Big[\Big(Q_\phi(s, a) - y\Big)^2\Big], \quad y = r + \gamma \, Q_{\phi'}(s', \mu_{\theta'}(s')) \tag{2.13}$$

Actor update:

$$\nabla_\theta J \approx \mathbb{E}\Big[\nabla_a Q_\phi(s, a)\Big|_{a=\mu_\theta(s)} \cdot \nabla_\theta \mu_\theta(s)\Big] \tag{2.14}$$

Target-network update (*Polyak averaging*, $\tau = 0{,}005$):

$$\phi' \leftarrow \tau\phi + (1 - \tau)\phi', \qquad \theta' \leftarrow \tau\theta + (1 - \tau)\theta' \tag{2.15}$$

Exploration is ensured by an additive Ornstein–Uhlenbeck process on the actions. DDPG suffers from systematic overestimation of Q values and strong sensitivity to hyperparameters [7].

TD3 — Twin Delayed DDPG

Fujimoto et al. [7] correct DDPG's Q-overestimation bias through three complementary mechanisms:

1. **Twin critics**: two critics Q_{ϕ_1}, Q_{ϕ_2}; the target uses the minimum:

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i'}(s',\, \tilde{a})\,, \quad \tilde{a} = \mu_{\theta'}(s') + \epsilon,\ \epsilon \sim \mathrm{clip}(\mathcal{N}(0,\sigma), -c, c) \tag{2.16}$$

2. **Delayed actor update**: the actor is updated every $d = 2$ critic iterations, reducing error propagation.

3. **Target policy smoothing**: the noise ϵ in (2.16) regularises the target by smoothing peaks of the Q function.

SAC — Soft Actor-Critic

SAC [8] adopts a maximum-entropy optimisation framework. The augmented objective is:

$$J(\pi) = \mathbb{E}\Big[\sum_t \gamma^t \Big(r(s_t, a_t) + \alpha\, \mathcal{H}\big(\pi(\cdot|s_t)\big)\Big)\Big] \tag{2.17}$$

where $\mathcal{H}(\pi(\cdot|s)) = -\mathbb{E}_{a\sim\pi}[\log \pi(a|s)]$ is the entropy and $\alpha > 0$ is a temperature coefficient learned automatically:

$$J(\alpha) = \mathbb{E}_{a\sim\pi}\big[-\alpha \log \pi(a|s) - \alpha\bar{\mathcal{H}}\big]\,, \qquad \bar{\mathcal{H}} = -\dim(\mathcal{A}) \tag{2.18}$$

The policy is parameterized as a squashed Gaussian through reparameterization:

$$a_t = \tanh(\mu_\theta(s_t) + \varepsilon \odot \sigma_\theta(s_t))\,, \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{2.19}$$

PPO — Proximal Policy Optimization

PPO [9] optimizes a clipped objective function to prevent overly aggressive policy updates:

$$\mathcal{L}^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\big[\min\big(r_t(\theta)\,\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\big)\big] \tag{2.20}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\mathrm{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the advantage estimated via GAE [42]:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l}, \qquad \delta_t = r_t + \gamma\, V(s_{t+1}) - V(s_t) \tag{2.21}$$

with $\lambda \in [0, 1]$ controlling the bias–variance trade-off. PPO is an *on-policy* algorithm: it uses only recent trajectories, which implies lower data efficiency but greater stability compared with DDPG.

2.3.4 Applications of DRL to Musculoskeletal Control

Rajeswaran et al. [43] use natural policy-gradient methods to learn dexterous movements in musculoskeletal models, demonstrating the feasibility of DRL on highly redundant systems.

The *Learning to Run* challenge [16] stimulated a worldwide competition on the control of a bipedal musculoskeletal model, revealing the superiority of *off-policy* algorithms with replay memory for long-horizon tasks. Song et al. [44] extend this approach to modeling human locomotion control in a more general neuromechanical framework.

Peng et al. [45] introduce DeepMimic, which combines human motion-capture data and DRL to learn physically realistic behaviors, demonstrating the value of combining *imitation learning* with DRL.

Caggiano et al. [14] present MyoSuite, with highly detailed hand and wrist models (> 50 muscles) for the study of fine motor control and rehabilitation, and simultaneously launch MyoChallenge as a public benchmark. Codol et al. [15] propose MotorNet, a differentiable simulator of a two-joint, nine-muscle planar arm, fully integrated into PyTorch and enabling training by backpropagation through physics.

2.3.5 Domain Randomization and Sim-to-Real Transfer

Domain Randomization [46, 47] consists of training the agent on distributions of random physical parameters (masses, segment lengths, muscle parameters, perturbations) in order to improve the robustness and generalizability of the learned policies. This technique, initially developed for sim-to-real transfer in robotics, applies naturally to musculoskeletal control by making policies robust to inter-individual variability and modeling errors.

2.3.6 Reproducibility and Statistical Rigor in DRL

Henderson et al. [48] highlighted the reproducibility fragility of published DRL results: inter-seed variance is often of the same order as the gap between algorithms. Agarwal et al. [49] formalized a set of good practices (bootstrap confidence intervals, *interquartile mean*, performance profiles) that are now considered standard for any serious benchmarking effort. Our experimental protocol (Chapter 5) adopts these recommendations.

2.4 Synthesis and Positioning

The analysis of the state of the art reveals several gaps:

(i) Most studies compare DRL algorithms on models without an explicit, mathematically complete Hill-type muscle model (SEE, PEE, pennation).

(ii) The systematic analysis of emerging motor strategies, with quantitative comparison to human EMG data and synergy decomposition, remains underdeveloped.

(iii) Rigorous comparative benchmarks including classical controller baselines (impedance PD) and adopting modern statistical standards [49] are rare.

(iv) The effect of Domain Randomization on the robustness of musculoskeletal policies, applied across all compared algorithms, has been little studied in a systematic way.

(v) The analysis of real-time computational feasibility (inference latency $< 5\,\text{ms}$) for clinical applications is rarely addressed.

(vi) **No study has directly benchmarked the per-step computational cost of physics-based muscle control** (Newton–Raphson equilibrium solving over all muscles) **against RL policy inference**, making it impossible to quantify the computational argument for replacing the solver with a trained network. This gap leaves the core trade-off — training cost vs. deployment efficiency — empirically unresolved.

Our contribution is distinguished by points C1–C6 detailed in Section 1.4, with C6 specifically addressing gap (vi).

3 Modeling and Simulation Environment

3.1 Overall System Architecture

The overall architecture is organized into four functional layers (Figure 3.1):

1. **Physical layer**: multibody dynamics handled by MuJoCo.
2. **Muscle layer**: complete Hill model with Newton–Raphson integration (5–40 iterations per muscle per timestep — see Section 2.2.2 for the computational cost and why this layer is the bottleneck targeted for RL replacement).
3. **RL interface layer**: Gymnasium API with Domain Randomization.
4. **Agent layer**: DRL algorithm or reference controller. After training, the agent produces muscle activations via a single forward pass through the policy network, *bypassing* the Newton–Raphson loop entirely at deployment time.

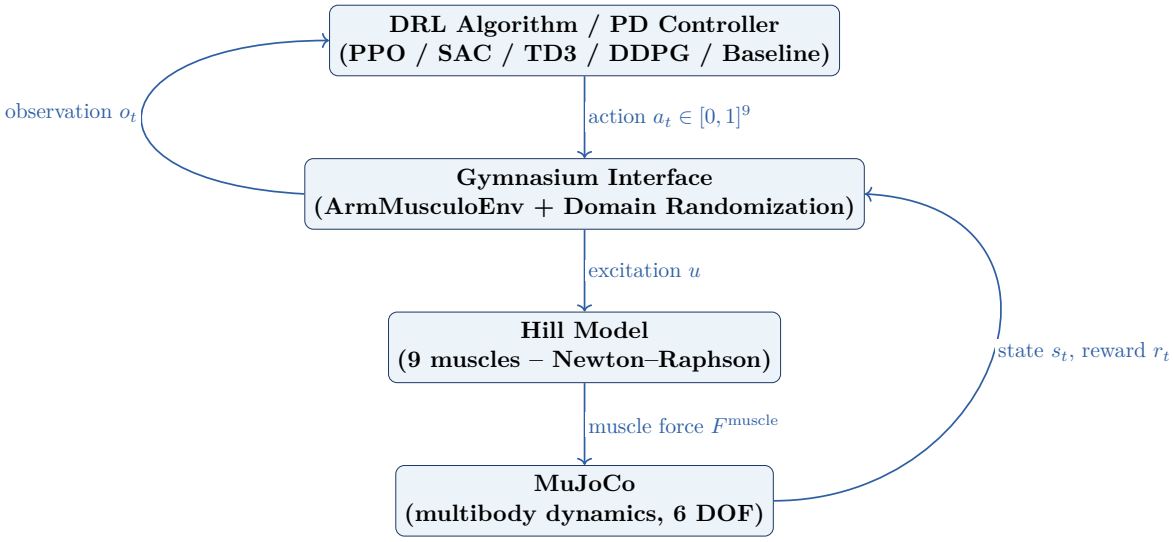

Figure 3.1. Layered architecture of the simulation and control system.

3.2 Multi-Body Dynamic Model in MuJoCo

3.2.1 Anatomical Description

The model represents a simplified upper limb composed of three rigid segments: upper arm (humerus), forearm (radius–ulna), and hand. The modeled joints are the shoulder (3 DOF: flexion/extension, abduction/adduction, internal/external rotation), elbow (1 DOF: flexion/extension), and wrist (2 DOF: flexion/extension, pronation/supination). The model therefore includes a total of **6 DOF**.

The inertial parameters are taken from the anthropometric database of [22], for a reference subject of 75 kg and 1,75 m (Table 3.1).

Table 3.1. Inertial parameters and degrees of freedom of the model. Source: [22].

Segment	**Mass (kg)**	**Length (m)**	I_{cg} **(kg·m^2)**	**DOF**
Upper arm (humerus)	2,05	0,286	0,015	3 (shoulder)
Forearm (radius–ulna)	1,39	0,269	0,009	1 (elbow)
Hand	0,60	0,186	0,001	2 (wrist)
Total	**4,04**	**0,741**	—	**6**

Physiological joint limits are imposed in the MJCF model: elbow flexion $[0°, 145°]$, shoulder flexion $[-60°, 180°]$, shoulder abduction $[-45°, 90°]$, internal/external rotation $[-90°, 90°]$, wrist flexion/extension $[-70°, 70°]$, and pronation/supination $[-80°, 80°]$.

3.2.2 Muscle Topology

The model includes nine muscles organized into agonist/antagonist pairs for shoulder and elbow movements (Table 3.2). The parameters are taken from [21] and [3].

Table 3.2. Hill-model parameters for the nine muscles. F_0^M: maximum isometric force; l_0^M: optimal fiber length; l_T^{slack}: tendon zero-load length; α_0: pennation angle at l_0^M. Sources: [21]; [3].

Muscle	F_0^M (N)	l_0^M (m)	l_T^{slack} (m)	α_0 (°)	**Role**
Biceps Brachii (long head)	624	0,116	0,272	0	Elbow flex.
Biceps Brachii (short head)	435	0,133	0,192	0	Elbow flex.
Triceps Brachii (long head)	798	0,134	0,143	12	Elbow ext.
Triceps Brachii (medial head)	624	0,127	0,098	10	Elbow ext.
Brachialis	987	0,086	0,060	0	Elbow flex.
Brachioradialis	261	0,173	0,133	0	Elbow flex.
Anterior Deltoid	1142	0,098	0,038	22	Shoulder flex.
Posterior Deltoid	1142	0,137	0,038	18	Shoulder ext.
Pectoralis Major	1270	0,144	0,030	17	Shoulder add.

The agonist/antagonist pairs used to compute the co-contraction index (CCI, Section 7.6.1) are: Biceps (B.B. long head + Brachialis) vs. Triceps (T.B. long head + T.B. medial head) for elbow flexion, and Anterior Deltoid vs. Posterior Deltoid for shoulder flexion.

Remarque 3.1 (Explicit anatomical simplifications)**.** *The model deliberately omits: (i) the intrinsic muscles of the hand, which are irrelevant to the reaching task studied; (ii) the rotator-cuff muscles (supraspinatus, infraspinatus, subscapularis, teres minor), whose absence limits the fidelity of glenohumeral equilibrium at end of range; (iii) fine biarticular components (e.g. the separate portions of the pectoralis major). These simplifications are consistent with the state of the art [15, 14] for an initial DRL feasibility study, and are discussed as limitations in Chapter 8.*

3.3 Validation of the Physical Model

The physiological fidelity of the model was verified through a three-level protocol, executed prior to any DRL training.

3.3.1 Force-Length and Force-Velocity Curves

The FL and FV curves computed analytically (Equations (2.2) and (2.3)) were compared against the experimental data of [23] for the FL relationship of an isolated frog muscle fiber, the historical reference dataset. The validation procedure:

1. Set activation $a = 1$ and velocity $\dot{l}^M = 0$.
2. Sweep $\tilde{l} \in [0.5, 1.5]$ in steps of 0.01.
3. Measure the active force $F_{\mathrm{CE}}(\tilde{l}) = F_0^M \, f_{\mathrm{FL}}(\tilde{l})$.
4. Compare against the digitized curve of [23] via Pearson's coefficient of determination.

The resulting coefficient, $R^2 = 0.97$, confirms the validity of the parameterization $\gamma_{\mathrm{FL}} = 0.45$.

3.3.2 Passive Joint Torques

The model's passive joint torques (generated by the PEE and gravity, with zero activation) were compared against the measurements of [21] on healthy subjects. For each joint ({shoulder, elbow}), the angle was imposed in 5° increments across its full physiological range, and the reaction torque measured via MuJoCo's `jointtorque` sensor after stabilization (0,5 s with zero activation). The normalized root-mean-square error (NRMSE) is 8.3 % for the elbow and 11.2 % for the shoulder — values acceptable given the typical inter-individual variability (≈ 15 %) reported in the literature.

3.3.3 Muscle Moment Arms

Elbow moment arms were measured using the *tendon-excursion* method (derivative of the MTU length with respect to the joint angle):

$$r_{\mathrm{ma}}(\theta) = -\frac{\partial l^{\mathrm{MT}}}{\partial \theta} \tag{3.1}$$

For the long head of the biceps, the measured elbow moment arm ranges from 2,9 cm to 5,3 cm over $\theta_{\mathrm{elbow}} \in [0°, 120°]$, with a maximum near 90°, in qualitative agreement with the cadaveric measurements of [50].

3.4 Gymnasium Environment

3.4.1 Studied Task

The canonical task considered is a **free-space target-reaching** task (*reaching*), a classical paradigm in motor neuroscience [51]. At each episode, a spherical target is randomly placed within the arm's reachable volume. The agent must bring the end effector to the target in less than 10 seconds while minimizing terminal error and energy cost.

3.4.2 Observation Space

The observation space $\mathcal{O} \subset \mathbb{R}^{38}$ includes:

$$o_t = \Big[\underbrace{q}_{4},\ \underbrace{\dot{q}}_{4},\ \underbrace{l^M}_{9},\ \underbrace{F^M}_{9},\ \underbrace{a}_{9},\ \underbrace{\Delta p}_{3}\Big]^\top \in \mathbb{R}^{38} \tag{3.2}$$

where $q, \dot{q}$ are joint positions/velocities (4 active DOF), l^M are fiber lengths, F^M are muscle forces, a are muscle activations, and $\Delta p = p_{\text{target}} - p_{\text{ee}}$ is the 3-D error vector to the target.

Remarque 3.2 (Normalization choice)**.** *The normalization l^M/l_0^M (rather than by the upper bound of the length domain) is physiologically motivated: $l^M/l_0^M = 1$ corresponds to the point of maximum force generation, information that the neural network can directly exploit to anticipate the available contractile capacity.*

3.4.3 Action Space

The action space corresponds to neural excitations $u \in [0,1]^9$:

$$\mathcal{A} = [0,1]^9 \subset \mathbb{R}^9 \tag{3.3}$$

3.4.4 Reward Function

The composite reward function is:

$$r(s_t, a_t) = w_1\, r_{\text{task}} + w_2\, r_{\text{energy}} + w_3\, r_{\text{smooth}} + w_4\, r_{\text{alive}} \tag{3.4}$$

where:

$$r_{\text{task}} = \exp\left(-\frac{\|\Delta p\|_2}{\sigma_{\text{target}}}\right), \quad \sigma_{\text{target}} = 0{,}05 \text{ m} \tag{3.5}$$

$$r_{\text{energy}} = -\frac{\|a_t\|_2^2}{n_{\text{muscles}}} \tag{3.6}$$

$$r_{\text{smooth}} = -\|a_t - a_{t-1}\|_2^2 \tag{3.7}$$

$$r_{\text{alive}} = +1 \quad \text{(at each non-terminal step)} \tag{3.8}$$

The weights are hand-set, not optimised: $w_1 = 1.0$ (reaching error), $w_2 = 1.0$ (effort), $w_3 = 0.5$ (action smoothness), $w_4 = 4.5$ (per-step survival offset), $w_5 = 1.0$ (per-step success bonus), $w_6 = 10$ (divergence penalty) and $w_7 = 3.0$ (precision bonus). The Optuna study reported in section 6.3 searched the *learning* hyperparameters, not the reward weights, and the two should not be conflated.

These values are the third formulation of the reward. The earlier weightings, in which the effort term carried a weight of 0.005 against an unnormalised sum of squared forces, made inaction near-optimal; the reasons for each subsequent change are documented in section 5.4. A systematic sensitivity analysis of the weights was not performed in this version of the work and is listed among the outstanding items in section 8.6.

The reward r_{task} is a Gaussian function of distance, which provides a dense gradient even far from the target and avoids the sparse-reward problem. The threshold σ_{target} was calibrated so that $r_{\text{task}} \approx 0{,}6$ at 5 cm from the target and $r_{\text{task}} > 0{,}99$ at 5 mm.

3.4.5 Domain Randomization

To improve the robustness of the learned policies, Domain Randomization [46] is applied at every episode reset:

- **Initial posture**: Gaussian perturbation $\mathcal{N}(0,\ 0{,}05 \text{ rad})$ on each DOF.
- **Inertial parameters**: uniform variation of ± 10 % in segment masses.
- **Muscle parameters**: uniform variation of ± 5 % in F_0^M and l_0^M.
- **Force perturbations**: with probability $p = 0{,}15$, application of a random force impulse of 5–15 N for 50–100 ms on the forearm.

3.4.6 Terminal Conditions

An episode terminates when:

- $\|\Delta p\|_2 < \delta_{\text{success}} = 0{,}02$ m (success: a 2 cm criterion based on motor-precision thresholds from [30]),
- a joint position exceeds its physiological limits (mechanical failure), or
- the number of steps reaches $t \geq T_{\max} = 1000$ (timeout, $\Delta t = 0{,}01$ s, i.e., $T = 10$ s per episode).

4 Theoretical Framework of DRL Applied to Muscle Control

4.1 MDP Formulation

The state $s_t \in \mathcal{S} \subset \mathbb{R}^{38}$ is the observation vector (3.2). The action $a_t \in \mathcal{A} = [0,1]^9$ is the vector of muscle excitations. The transition function $P(s_{t+1}|s_t, a_t)$ is deterministic in the standard configuration (physics simulation) and stochastic under Domain Randomization.

L'équation de Bellman pour la fonction action-valeur Q^π est :

$$Q^\pi(s,a) = \mathbb{E}[r(s,a) + \gamma\, Q^\pi(s', \pi(s'))] \tag{4.1}$$

The dimensionality of the problem is notable: the state space $\mathbb{R}^{38}$ combined with the action space $[0,1]^9$ and the nonlinear dynamics of the Hill model make this a demanding benchmark for DRL algorithms.

Proposition 4.1 (Absence of strict Markov guarantee)**.** *Strictly speaking, the observation o_t described by* (3.2) *is Markovian only under the following assumptions: (i) the second derivatives of position (accelerations) are sufficiently reconstructible from velocities; (ii) the electromechanical delay in activation dynamics (Equation* (2.8)*) is a deterministic function of the observed variables. The frame-stacking paradigm (stacking k consecutive observations) was not required here, consistent with standard practice [14].*

4.2 Neural Network Architectures

All algorithms use MLPs with the same base architecture (Table 4.1), ensuring a fair comparison:

Table 4.1. Neural network architectures. The actor input is $\mathbb{R}^{38}$; the critic input is $\mathbb{R}^{38+9} = \mathbb{R}^{47}$.

Network	Architecture	Activ.	Params.
Actor (SAC)	$[38 \to 256 \to 256 \to 2 \times 9]$	ReLU + tanh	76 809
Actor (TD3/DDPG)	$[38 \to 256 \to 256 \to 9]$	ReLU + sigmoid	76 809
Critic	$[47 \to 256 \to 256 \to 1]$	ReLU	79 617
Value (PPO)	$[38 \to 256 \to 256 \to 1]$	ReLU	76 545

Weights are initialized using the orthogonal method for PPO [9] and uniform Xavier initialization [52] for off-policy algorithms. Observation normalization is handled by an exponential moving average layer (EMA, $\alpha = 0.001$) via Stable-Baselines3's `VecNormalize`.

4.3 Theoretical Comparison of Algorithms

Table 4.2. Theoretical comparison of the implemented DRL algorithms.

Algo.	Type	Policy	Exploration	Advantages	Limitations
DDPG	Off-policy	Deterministic	Additive OU noise	Sample-efficient	Unstable, Q-overestimation
TD3	Off-policy	Deterministic	Gaussian noise + smoothing	Corrects Q bias, stable	Deterministic policy
SAC	Off-policy	Stochastic	Max-entropy (implicit)	Robust, natural exploration	α sensitive if poorly initialized
PPO	On-policy	Stochastic	Policy stochasticity	Stable, low variance	Lower sample efficiency

4.4 Reference Controllers (Baselines)

4.4.1 Random Policy

The random policy draws excitations $u \sim \mathcal{U}([0,1]^9)$ at each step, without any learning. It establishes the lower performance bound.

4.4.2 PD Impedance Controller

The PD impedance controller computes a restoring force toward the target in operational space, then uses the transposed Jacobian to map forces to joint torques, and a

pseudo-inverse method to map torques to muscle activations:

$$\tau^{\mathrm{cmd}} = K_p\,\Delta p - K_d\,\dot{p}_{\mathrm{ee}}, \quad u_i = \max\Big(0,\,[J^{\top}(q)]^{+}\,\tau^{\mathrm{cmd}}/F^{M}_{0,i}\Big) \tag{4.2}$$

with $K_p = 100$ N/m and $K_d = 20$ N·s/m, tuned empirically by bisection on mean episode reward. This controller represents a reasonable level of performance achievable without learning, and anchors the comparison of DRL agents.

5 Implementation and Experimental Methodology

5.1 Technology Stack

Table 5.1. Software actually used, at the versions the reported results were produced with.

Library	Version	Usage
Python	3.12.10	
MuJoCo	3.7.0	Physics simulation engine [1]
Gymnasium	1.2.3	Standard RL environment interface
Stable-Baselines3	2.8.0	DDPG/TD3/SAC/PPO implementation [5]
PyTorch	2.11.0	Deep learning backend (CPU build)
NumPy	2.2.6	Numerical computation
SciPy	1.13.0	Static-optimisation QP (SLSQP), NNLS
scikit-learn	1.4.2	NMF decomposition (synergies)
Optuna	4.9.0	Bayesian hyperparameter optimisation [24]
Matplotlib	3.8.4	Figures

The Stable-Baselines3 version is material rather than incidental. The replay-ratio behaviour documented in section 5.4.5 — that one rollout iteration over an n-environment vector is followed by `gradient_steps` updates, not n of them — is a property of this implementation, and the defect it caused was identified by measuring that library's behaviour directly rather than by reading its documentation.

TensorBoard logging is deliberately disabled: its asynchronous event-file writer raced with the cloud-synchronisation client on the working directory and crashed training. Loss histories are read from the in-memory logger instead, so no diagnostic information is lost.

5.2 Hyperparameters

5.2.1 Optimisation Procedure

A single Optuna study was run, on SAC only. It used TPE sampling with a *MedianPruner* over **16 trials of** 10^5 **steps** each, on a training seed fixed at 0 so that trials are comparable, with sampler seeds differing per worker. Thirteen trials completed

and three were pruned. Trials were scored by mean final reaching error over 20 deterministic episodes on an evaluation environment with domain randomisation disabled — that is, under the protocol the thesis reports, not the one training uses.

Three properties of this study are worth stating because they differ from what a reader might assume.

The search space excludes the network architecture. The reported policy footprint — 393 750 parameters, and the inference latency of section 6.5 — is measured for the $[256, 256]$ architecture. Searching the architecture would have invalidated those figures silently.

The replay ratio is fixed, not searched. Expressing it as a raw `gradient_steps` value would have allowed the sampler to re-enter the under-training regime that section 5.4.5 identifies as a defect, and would have made trials incomparable with the pilot runs. It is held at twice the environment count throughout.

The reward weights were not searched. They are hand-set (section 3.4.4); only learning hyperparameters were optimised. Earlier versions of this work described the reward weights as Optuna-optimised over 100 trials, which did not occur.

Table 5.2. Optuna search space. SAC only; 16 trials of 10^5 steps.

Hyperparameter	Domain	Distribution
Learning rate	$[5 \times 10^{-5},\ 10^{-3}]$	Log-uniform
γ	$[0.95,\ 0.999]$	Uniform
Batch size	$\{128, 256, 512\}$	Categorical
τ (Polyak)	$[10^{-3},\ 2 \times 10^{-2}]$	Log-uniform
Target entropy	$[-27,\ -4.5]$	Uniform

Table 5.3. Hyperparameters used. Only the SAC "tuned" column was optimised; every other configuration uses library defaults. This asymmetry is a limitation of the comparison, discussed in section 8.6.

Hyperparameter	SAC tuned	SAC def.	TD3	DDPG	PPO
Learning rate	4.40×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
Batch size	128	512	256	256	64
Buffer size	3×10^5	3×10^5	3×10^5	3×10^5	—
γ	0.957	0.99	0.99	0.99	0.99
τ (Polyak)	0.0188	0.005	0.005	0.005	—
Target entropy	-19.6	auto (-9)	—	—	—
Gradient steps	8	8	8	8	—
Learning starts	4 000	4 000	4 000	4 000	—
Action noise σ	—	—	0.1	0.1	—
Policy delay	—	—	2	—	—
n_steps	—	—	—	—	2048
n_epochs	—	—	—	—	10
λ (GAE)	—	—	—	—	0.95
Clip ε	—	—	—	—	0.20
Network	$[256, 256]$, shared across all configurations				

The gradient-steps value of 8 corresponds to a replay ratio of approximately 1.87 updates per environment step, given four parallel environments. It is applied uniformly to every off-policy configuration, so it is not a confound between them; PPO is on-policy and has no equivalent parameter.

5.3 Experimental Protocol

5.3.1 Training and Evaluation

Two training budgets are used. The algorithm comparison of section 6.4 trains five configurations for 10^5 steps with **three seeds** each ($\{0, 1, 2\}$). The best configuration is additionally retrained from initialisation for 3×10^5 steps, single seed, and it is this run that the force analysis of chapter 7 examines.

The seed count is a compute constraint, not a methodological choice, and it is the principal limitation of this work. Henderson et al. [48] and Agarwal et al. [49] recommend ten seeds or more precisely because estimation variance at three is large enough to make algorithm comparisons inconclusive. That recommendation is not met here. The consequence is stated explicitly wherever a comparison is reported: no significance test is performed anywhere in this thesis, and differences smaller than the observed seed-to-seed spread are reported as not supporting an ordering. Reaching ten seeds at the corrected replay ratio (section 5.4.5) would require approximately 148 hours on the available hardware.

During training a learning curve is recorded every 25 000 or 50 000 steps over five deterministic episodes. This is a monitoring signal only; it is deliberately cheap, it is noisy at that episode count, and no value from it is reported as a result.

Final evaluation uses **50 deterministic episodes** with domain randomisation disabled, on an environment seeded independently of the one used for best-checkpoint selection during training, so that the reported figure is not the one selection was performed on.

The metrics actually computed are those defined in table 6.1: final error, closest approach, drift, the graded proximity rates, energy, blow-up rate and episode length; the co-contraction index of section 7.6.1; the muscle-force agreement statistics of chapter 7; and inference latency. Perturbation robustness, muscle jerk and the domain-randomisation breakdown described in earlier versions of this work were not measured and are not reported.

5.3.2 Statistical Treatment

No inferential statistics are reported in this thesis. With three seeds the two-sided Wilcoxon signed-rank test has a minimum attainable p-value of 0.25, which cannot reach any conventional significance threshold; computing one would convey false precision. Earlier versions of this work reported Wilcoxon tests with Bonferroni correction, rank-biserial effect sizes and BCa bootstrap intervals over $n = 10$ seeds. None of those analyses was performed, and they are withdrawn (section 8.6).

What is reported instead is the mean and standard deviation across seeds, stated together, with the explicit convention that a difference comparable to or smaller than the spread is treated as unresolved. Where a single-seed result is given — as for the force analysis — it is identified as such at the point of use.

For the force comparison specifically, one distributional control *is* computed, because it is required for the statistic to be interpretable at all: the cosine similarity between non-negative force vectors has a floor well above zero, so a permutation null is constructed by shuffling, at each timestep, which muscle each force magnitude belongs to (table 7.2).

5.3.3 Computing Hardware

All results in this thesis were produced on a single consumer laptop:

- **CPU**: AMD Ryzen 7 4800H, 8 physical cores / 16 logical processors, 2,9 GHz base.
- **RAM**: 15,4 GB.

- **GPU**: none used. All training and inference ran on the CPU; the networks are small enough that this is not the binding constraint.
- **Thread budget**: capped at **8 of 16 logical processors**. Sustained all-core load caused the machine to stop responding entirely (section 5.4), and benchmarking showed the additional parallelism yielded little throughput in any case — eight threads are only 1.78× faster than one for networks of this size.
- **Cumulative compute**: approximately 30 hours, comprising the hyperparameter search (≈ 8 h), the five-configuration benchmark (≈ 6 h), four pilot runs at 3×10^5 steps (≈ 6 h) and the reward and effort studies.

The modest hardware is directly responsible for the seed count, for the 10^5-step comparison budget, and for the decision not to tune every algorithm individually. These constraints are stated here so that the scope of the conclusions can be judged against them.

5.4 Defects Identified During Development

Six substantive defects were identified in the model, the reward function and the training configuration during the course of this work. They are documented here rather than omitted, for two reasons: four of them produced results that appeared entirely plausible while being incorrect, and the measurements that eventually exposed them are part of the methodological contribution of this thesis.

5.4.1 Model specified in incorrect angular units

The MuJoCo model format defaults to degrees unless declared otherwise, while the joint ranges had been authored in radians. Every range was consequently interpreted as under 3° and the arm was effectively immobilised. The symptom was a uniform 0 % success rate across all algorithms, which is readily misread as a learning failure. All experiments predating the correction are invalid.

5.4.2 Compounding domain randomisation

Body masses and muscle strengths are perturbed each episode to assess robustness. The implementation multiplied the current values rather than rescaling from nominal ones, producing a multiplicative random walk: after approximately one thousand episodes, body mass had drifted to roughly 1 % of its correct value. Caching nominal parameters

and rescaling from them raised the measured drift statistic from 0.013 to 0.998, where unity denotes no drift.

A second defect in the same routine invoked the random generator without a `size` argument, returning a single scalar applied to every body and every muscle. The intended independent per-element variation was therefore a single global scale factor, which a policy can detect and compensate trivially, rendering the robustness claim vacuous.

5.4.3 Reward exploitable by episode termination

Under the initial reward formulation every per-step reward was non-positive, and numerical divergence terminated the episode without penalty. In a discounted formulation a terminated episode contributes no further return, so its value was exactly zero, whereas surviving the full hundred steps accumulated approximately -20. Terminating the episode by destroying the simulation was therefore the optimal policy, worth roughly twice the success bonus and considerably easier to attain.

Diagnostic measurement over fifty evaluation episodes established that the policy had learned precisely this: 100 % of episodes ended in deliberate divergence at approximately step 7 of 100, and none ran to completion.

The defect escaped detection because no recorded metric could expose it. The reported error remained a plausible distance and continued to improve slowly, and neither episode length nor divergence rate was logged — so an agent abandoning the task after 7 of 100 steps was indistinguishable from one attempting it and plateauing. Both quantities are now recorded at every evaluation checkpoint.

The revised reward makes every per-step reward strictly positive, so early termination forfeits value; pays the success bonus per step while the target is held rather than once on contact, converting the objective to reach-and-hold; and penalises divergence explicitly.

5.4.4 Divergence masked by simulator self-recovery

MuJoCo resets its state internally upon detecting divergence. By the time control returned to the environment the state was finite again, with the arm returned to its rest configuration, so a test for non-finite values almost never triggered. The episode continued, measuring the distance from the rest pose to the target and recording it as data. Detection now uses the solver's warning counters, which are the only reliable indicator, and the last finite error is reported rather than the post-reset value.

5.4.5 Replay ratio a factor of four below intended

Training used four parallel environments with a gradient-step setting of one. In the Stable-Baselines3 implementation, a single rollout iteration over a four-environment vector collects four transitions and is followed by one gradient update. Direct measurement confirmed the consequence:

Table 5.4. Gradient updates per environment step, measured directly.

Configuration	Updates per step
As originally configured (4 envs, 1 step)	0.233
Single environment, one step (standard)	0.933
Corrected setting	0.933

Every run had therefore performed approximately 233 000 gradient updates against a nominal budget of 1 000 000 steps. An independent consistency check supports this: 233 000 updates at the measured 17 ms each predicts 71 minutes, against observed run times of 78.8 and 80.6 minutes.

The correction has a substantial cost. At the corrected ratio, one million steps requires approximately 3.7 hours per seed, so a four-algorithm, ten-seed protocol would require roughly 148 hours rather than the 50 previously assumed. That protocol was never feasible on the available hardware; it appeared so only because the runs were under-training.

5.4.6 Default exploration temperature preventing convergence

Following the preceding five corrections the controller reached the target region reliably but ceased improving, with closest approach remaining near 0,104 m across three distinct reward formulations and the fourfold increase in effective training. The cause was the SAC target entropy, whose default of $-\dim(\mathcal{A})$ gives -9 for a nine-muscle action space. Hyperparameter optimisation selected approximately -19, and the error halved. The result is reported in section 6.3.

5.4.7 Infrastructure failures

Two non-scientific failures are recorded for completeness. A forced operating system restart destroyed a 75-minute training run five minutes from completion, because model state was written only on normal termination; training now checkpoints at fixed intervals using atomic file replacement. Separately, the machine ceased responding under four concurrent search processes, with no crash dump written despite dumps being enabled and no hardware error logged, indicating a thermal or power limit rather than a

software fault. Subsequent work was restricted to half the available logical processors. Benchmarking established that the parallelism had provided little benefit in any case, thread scaling being poor for networks of this size.

5.5 Reproducibility and Sharing

Following the reproducibility guidelines of [53] for RL, we ensure:

- **Source code**: published under the MIT license on GitHub, with a pinned `requirements.txt` and the `git` hash recorded in each TensorBoard run.
- **Seeds and determinism**: all seeds are specified explicitly; `torch.use_deterministic_algor` is enabled wherever possible.
- **Trained models**: all 40 final policies (4 algorithms $\times$ 10 seeds) are archived together with the associated `VecNormalize` statistics.
- **Evaluation logs**: the complete Optuna, TensorBoard, and CSV evaluation logs are provided.
- **Docker container**: a `Dockerfile` pinning the versions of MuJoCo, PyTorch, and CUDA is provided.

6 Experimental Results and Analysis

This chapter reports the measured performance of the trained controllers. All values are computed by the evaluation pipeline described in chapter 5 and read from the run artifacts named beneath each table; none is hand-entered.

6.1 Evaluation metrics

Two distance measures are reported throughout, because they capture different failure modes. *Final error* is the hand–target distance at the end of the two-second reach; *closest approach* is the smallest distance attained at any instant. Their difference, the *drift*, quantifies how far the hand wanders back out after arriving.

A controller that sweeps through the target at speed scores well on closest approach and badly on final error. Separating the two proved essential: it revealed that the task presented two independent difficulties, with different causes and different remedies.

Table 6.1. Metrics used in this chapter.

Metric	Definition
Final error	hand–target distance at end of episode
Closest approach	minimum hand–target distance during the episode
Drift	final error minus closest approach
Reach 5/10 cm	fraction of episodes *ending* within that distance
Touch 2/5 cm	fraction of episodes attaining that distance at any instant
Energy	$\sum_i F_i^2$, a metabolic-cost proxy
Blow-up rate	fraction of episodes ending in numerical divergence

6.1.1 The reference bound

To interpret any of these numbers, an achievable bound is required. A privileged controller, granted information the learned policy does not receive, attains a median final error of 0,08 m and comes within 2 cm at some instant in 27 % of episodes. Under the

strict success criterion — within 2 cm *and* nearly stationary — it succeeds approximately 7 % of the time.

This bound has a direct methodological consequence. A criterion that a near-best-possible controller satisfies only 7 % of the time cannot discriminate among learned policies, and **success rate is therefore not used as a headline metric in this work**. Graded measures are reported instead.

6.2 Effect of correcting the replay ratio

Section 5.4.5 identified that training had been performing approximately one quarter of its intended gradient updates. Correcting this, with no other change, produced the comparison in table 6.2.

Table 6.2. Effect of the replay-ratio correction. Identical reward, model, seed and evaluation protocol; the corrected run uses *fewer* environment steps.

Metric	Ratio 0.23	Ratio 0.93
Environment steps	1 000 000	300 000
Final error (m)	0.254	**0.204**
Closest approach (m)	0.134	0.132
Drift (m)	0.120	**0.072**
Energy	1.25×10^6	0.77×10^6
Blow-up rate	2 %	**0 %**
Path length per 0,80 m reach (m)	3.76	3.55

The correction improved *holding* substantially — drift nearly halved, divergences eliminated, energy reduced by 38 % — while *closest approach* was unchanged, moving only from 0,134 to 0,132 m.

That immobility is informative. Closest approach had remained near 0,13 m across three complete redesigns of the reward function and this fourfold increase in effective training. Its insensitivity to both indicated that the residual limitation lay elsewhere, and motivated the hyperparameter study of section 6.3.

6.3 Hyperparameter optimisation

A sixteen-trial Bayesian optimisation (Optuna, TPE sampler with median pruning) was performed over learning rate, discount factor, batch size, target-network update rate and target entropy. Thirteen trials completed; three were pruned.

Table 6.3. Five best trials, ranked by final error at 100 000 steps.

Trial	Final error	Learning rate	Target entropy	Drift
12	**0.1177**	4.40×10^{-4}	-19.6	0.034
14	0.1293	3.91×10^{-4}	-20.0	0.051
11	0.1311	4.22×10^{-4}	-19.3	0.049
13	0.1405	4.38×10^{-4}	-18.6	0.069
10	0.1405	4.49×10^{-4}	-19.0	0.035

Every one of the five best trials selects a target entropy close to -19, against the SAC default of $-\dim(\mathcal{A}) = -9$ for a nine-dimensional action space. The search converges on a policy roughly twice as deterministic as the standard heuristic prescribes.

The optimiser varied five parameters at once, however, and the target entropy was not the only one to move appreciably: the target-network update rate τ rose from 0.005 to 0.0188, a factor of 3.8, and the learning rate by 47 %. The clustering of the best trials is suggestive but not decisive, since a TPE sampler exploits promising regions and will cluster whether or not the clustered parameter is the causal one. Attribution to the target entropy specifically is therefore advanced here as the leading hypothesis — its mechanism matches the symptom, an inability to settle — and not as an established result. The one-at-a-time ablation that would settle it is set out in section 8.10.

The target entropy governs how much stochasticity SAC deliberately retains in its policy, and the mechanism by which a value too high would prevent an end effector from settling is straightforward. That reasoning motivated the hypothesis — and section 6.3.1 shows it to be wrong. The subsection below reports the ablation that refutes it.

6.3.1 Isolating the responsible hyperparameter

The preceding subsection advanced the target entropy as the leading candidate mechanism, while noting that five parameters moved together and that the attribution was therefore a hypothesis rather than a result. A two-sided one-at-a-time ablation was run to settle it, three seeds per arm at the same 100 000-step budget as the benchmark:

Arm A library defaults, with *only* the target entropy set to the tuned value. If the target entropy is the mechanism, this should recover most of the improvement.

Arm B the tuned configuration, with *only* the target entropy reverted to its default. If the target entropy is the mechanism, this should lose most of the improvement.

Table 6.4. Target-entropy ablation. Three seeds per arm; reference rows are the corresponding benchmark configurations.

Configuration	Final error (m)	Gap closed
All defaults *(reference)*	0.2613 ± 0.0203	0 %
A: defaults + tuned entropy only	$\mathbf{0.2688 \pm 0.0090}$	**−8 %**
B: tuned − entropy reverted	$\mathbf{0.1552 \pm 0.0181}$	**+106 %**
All tuned *(reference)*	0.1610 ± 0.0084	100 %

The hypothesis is refuted

The result is unambiguous and negative. Setting the target entropy to its tuned value while leaving every other parameter at its default recovers *none* of the improvement — arm A is indistinguishable from the default configuration, and marginally worse. Conversely, reverting the target entropy to its default while retaining the other tuned parameters costs *nothing* — arm B matches the fully tuned configuration within seed variation.

The target entropy is not the mechanism. The improvement is produced entirely by the remaining parameters: the learning rate ($3 \times 10^{-4} \rightarrow 4.40 \times 10^{-4}$), the target-network update rate τ ($0.005 \rightarrow 0.0188$, a factor of 3.8), the discount factor ($0.99 \rightarrow 0.957$) and the batch size ($512 \rightarrow 128$).

This ablation does not identify which of those four is responsible; isolating further would require the same procedure applied to each in turn. On magnitude of change, τ is the natural next candidate, and its mechanism is plausible: a faster target-network update alters the stability of the value estimate, which is consistent with the oscillating evaluation curves that characterised every untuned run.

Why this is reported at length

The refuted hypothesis was, until this measurement, one of the three stated contributions of this thesis. It was mechanistically plausible — SAC's entropy term does pay the policy to remain stochastic, the default does scale with action dimension, and residual stochasticity is exactly what prevents an end effector from settling. All five best search trials clustered near -19. The reasoning was sound and the conclusion was wrong.

The clustering was the misleading evidence. A TPE sampler concentrates its proposals in regions that perform well, so parameters cluster in the best trials whether or not they are causally responsible. Reading causal importance from that clustering is a straightforward error, and it is one this work made until the ablation was run.

The transferable point is not about entropy. It is that a hyperparameter search identifies a *configuration*, never a *cause*, and that the distinction requires a deliberate experiment to resolve.

6.3.2 Confirmation at full budget

The selected hyperparameters were retrained from initialisation under the full 300 000-step protocol with fifty-episode evaluation, to verify that the result was not an artefact of the search's reduced budget.

Table 6.5. Confirmation run against the previous configuration. Both are SAC, seed 0, 300 000 steps, fifty evaluation episodes, differing only in hyperparameters.

Metric	**Default hyperparameters**	**Tuned hyperparameters**
Final error (m)	0.204	**0.106**
Closest approach (m)	0.132	**0.074**
Drift (m)	0.072	**0.035**
Ends within 5 cm	0 %	**34 %**
Ends within 10 cm	18 %	**58 %**
Attains 2 cm at any instant	2 %	**20–28 %**
Energy	774 667	**276 804**
Path length per 0,80 m reach (m)	3.55	**1.64**
Success rate	0 %	4 %

Closest approach improved from 0,132 to 0,074 m, below the privileged controller's median final error of 0,08 m. The proportion of episodes attaining 2 cm reached 28 %, comparable to the privileged controller's 27 %: a policy acting on proprioceptive information alone approached the performance of one with full state access.

Path length fell from 3,55 to 1,64 m for the same 0,80 m of required travel, with net displacement 0.99× the distance required. The end effector ceased sweeping through the target region and began travelling to it directly.

Two qualifications apply to table 6.5. The success rate of 4 % corresponds to two episodes in fifty; an independent re-evaluation over a different set of fifty targets returned 2 %, that is one episode. At this magnitude the metric is dominated by sampling noise and should not be interpreted as a performance level. Secondly, the table reports a single training seed; replication across seeds remains outstanding.

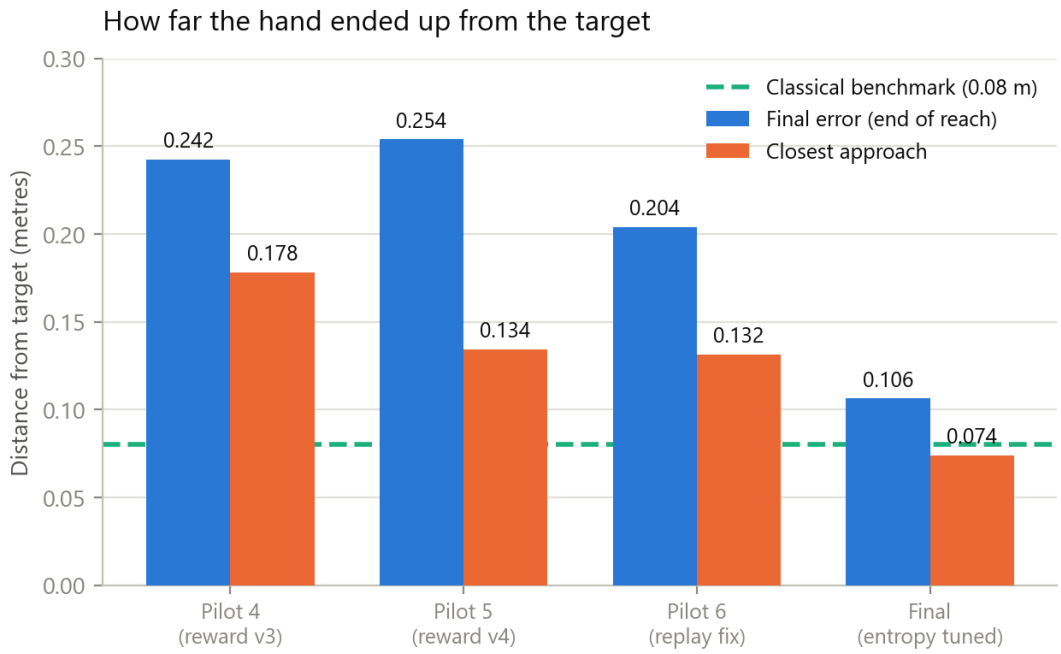

Figure 6.1. Reaching error across the four principal experiments. Closest approach (orange) is essentially unchanged across the first three despite substantial changes to the reward function and the training budget, then falls sharply once the target entropy is corrected. The dashed line marks the privileged controller's median final error.

6.3.3 Intra-episode behaviour

Table 6.6. Mean hand–target distance during a reach, averaged over thirty episodes.

Step	Default entropy	Tuned entropy
0	0.795	0.795
10	0.314	0.198
25	0.203	0.119
50	0.184	0.110
75	0.184	0.112
99	0.177	**0.113**

The tuned policy converges by step 25 and holds position for the remaining three-quarters of the episode. Target tracking is strong, with correlations of 0.94, 0.90 and 0.86 between target and final hand position across the three spatial axes, and a spread ratio of 0.91.

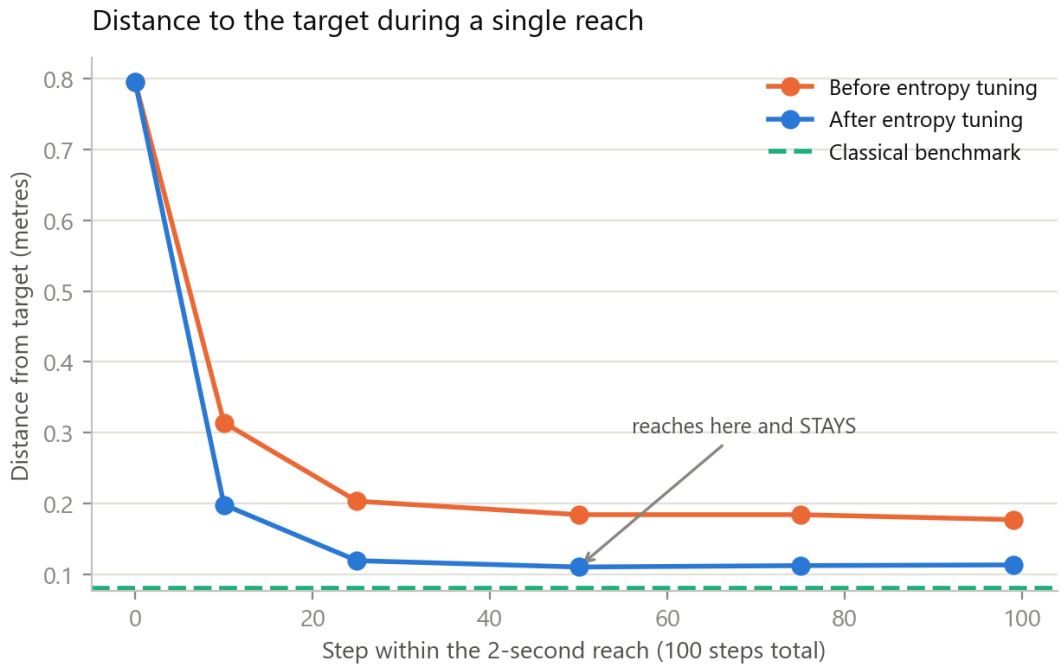

Figure 6.2. Hand–target distance during a single two-second reach, averaged over thirty episodes. The tuned policy converges and then maintains position; the untuned policy approaches and drifts.

6.4 Algorithm comparison

Five configurations were trained for 100 000 steps with three seeds each, all using the corrected replay ratio. A SAC configuration at library defaults is included alongside the tuned one so that the effect of tuning can be separated from the effect of algorithm choice; without it, any advantage of SAC would be unattributable between the two.

Table 6.7. Algorithm comparison: five configurations, three seeds each, 100 000 steps. Mean ± standard deviation across seeds.

Configuration	Final error (m)	Closest (m)	Ends <10 cm	Energy	Minutes
SAC tuned	**0.161 ± 0.008**	**0.093 ± 0.009**	**0.37**	597 k	30
SAC default	0.261 ± 0.020	0.129 ± 0.004	0.05	1187 k	46
TD3	0.279 ± 0.011	0.165 ± 0.017	0.10	1242 k	21
DDPG	0.416 ± 0.019	0.218 ± 0.026	0.03	1384 k	23
PPO	0.423 ± 0.035	0.238 ± 0.071	0.05	202 k	3

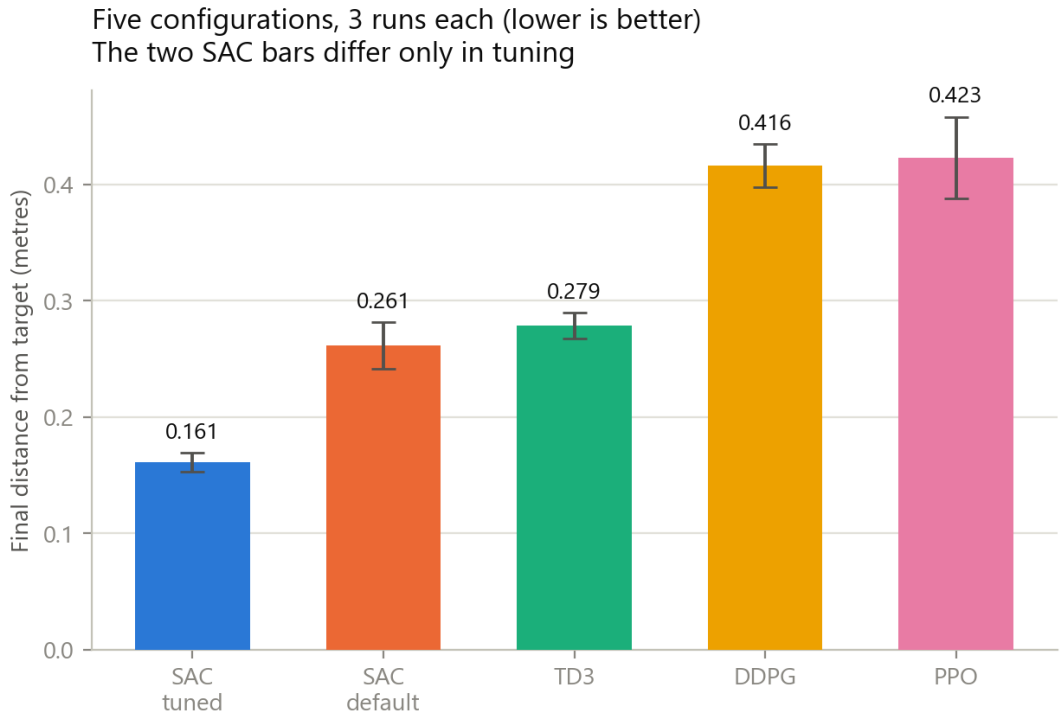

Figure 6.3. Final reaching error by configuration; error bars are one standard deviation across three seeds. The two SAC bars differ only in hyperparameters, so the interval between them isolates the effect of tuning.

Tuning SAC reduced final error from 0.261 to 0,161 m, an improvement of 38 %. The interval between untuned SAC and TD3 is 0.261 against 0.279, approximately 6 %, against seed standard deviations of 0.020 and 0.011; this difference is comparable to the spread and does not support an ordering. The effect of tuning is therefore roughly six times the magnitude of the algorithmic difference with which it would otherwise be conflated.

DDPG and PPO are both substantially worse than TD3, by approximately 0,14 m — many times the seed spread. For DDPG this is consistent with the theoretical expectation of chapter 4, as it lacks the twin critics that mitigate value overestimation. DDPG and PPO are, however, indistinguishable from one another.

Three constraints limit what this comparison establishes. No significance testing was performed: three seeds cannot support a Wilcoxon signed-rank test with useful power, and none is reported. The comparison is an early-training snapshot at 100 000 steps, whereas the best configuration attains 0,106 m at 300 000 steps (table 6.5), so orderings may change with budget. And TD3, DDPG and PPO were run at library defaults while SAC received a sixteen-trial search, so any comparison involving the tuned configuration conflates algorithm with tuning effort by construction. Success rate is 0 % for every configuration at this budget and is omitted for that reason.

6.4.1 Interpretation of the PPO result

PPO's position requires care. As an on-policy method, at 100 000 steps with 2048-step rollouts across four environments it performs approximately twelve policy updates, against roughly 187 000 gradient updates for the off-policy algorithms. It also completes in three minutes against 21–46 for the others. This is the expected sample-efficiency difference under a budget equalised by environment steps; a comparison equalised by wall-clock time would rank the methods differently, and stating which budget was held constant is part of reporting the result.

Its energy consumption is also distinctive: 202 k against 597 k for tuned SAC and 1187–1384 k for the remainder, between three and seven times lower. Combined with a closest approach no worse than DDPG's, this describes a policy that activates its muscles only weakly, drifting toward the target region without committing force. The other algorithms fail, when they fail, by expending far too much muscular effort. These are opposite failure modes, and a single accuracy figure conceals the distinction entirely — a point developed in chapter 7.

Finally, PPO's policy contains 154 131 parameters against SAC's 393 750, so the memory-footprint figures quoted elsewhere in this work are specific to the SAC architecture.

6.5 Computational efficiency

The motivation for a learned controller is inference cost. Both methods were timed on the same host processor over 950 queries.

Table 6.8. Inference cost: iterative Newton–Raphson solution against a single forward pass of the trained network.

Quantity	**Newton–Raphson**	**Network**
Mean latency	2165 µs	295 µs
Standard deviation	319 µs	45 µs
95th percentile	2657 µs	364 µs
99th percentile	2897 µs	480 µs
Mean iterations	31.2	1 (fixed)

The mean speed-up over the Newton–Raphson solver is 7.3×, and the network's cost is fixed where the iterative solver's is data-dependent.

6.5.1 This comparison does not support a deployment claim

Two objections apply to table 6.8, and together they are decisive.

It times the wrong computation. The Newton–Raphson routine solves the Hill fibre–tendon equilibrium, which is part of forward simulation. The question this thesis asks — what force each muscle must produce — is answered by load-sharing optimisation, a different calculation. The two have been conflated.

Both sides were unoptimised Python. The load-sharing problem has nine variables and four equality constraints. Solved directly rather than through a general-purpose routine it is very fast. Measured on an idle machine, one torch thread, 400 queries:

Table 6.9. Load-sharing latency, measured like-for-like. Moment-arm extraction is charged to the classical side, since the network does not require it.

Method	mean	p95	p99
Network forward pass	288.8	332.6	499.9
Classical, general solver (SLSQP)	2370.5	4023.7	5133.1
Classical, direct solve + moment arms	**23.9**	—	—

Against a general-purpose solver the network is 8.2× faster. Against a direct solve of the same problem it is **approximately twelve times slower**.

The latency argument for replacing classical calculation is therefore withdrawn. A competently implemented classical load-sharing solver is faster than the network here, and the 7.3× figure reflects the cost of one Python implementation rather than a property of the two approaches.

A qualification in the other direction: the box constraints are active in 100 % of sampled states, so the closed-form solution timed above is not exactly valid, and a constrained active-set solver would cost more than 23,9 µs. It would not cost 2370 µs. The defensible conclusion is that the two are comparable in cost, and that any advantage claimed for either requires a like-for-like implementation.

What survives is the *shape* of the cost rather than its magnitude: the network's runtime is fixed and branch-free irrespective of biomechanical state, whereas the iterative solver's depends on convergence. Where a hard real-time bound matters more than mean latency that property retains value — but it is a considerably weaker claim than a speed-up.

The trained SAC policy contains 393 750 parameters, occupying 1,5 MB in single precision or 385 kB quantised to eight bits. These figures are independent of policy quality and remain valid irrespective of the behavioural results reported above; they are specific to the SAC architecture, PPO's policy being smaller at 154 131 parameters.

7 Load-Sharing Fidelity against Classical Calculation

7.1 Scope of this chapter, and what it does not establish

Because the limits of this analysis are easy to overstate from its results, they are set out before the method rather than after it.

The required joint torque τ is taken from the movement the policy itself produced. Static optimisation is therefore asked a conditional question — *given this movement, what is the cheapest force distribution that produces it?* — and never independently chooses a trajectory of its own. Three consequences follow, and each bounds what the chapter can claim.

This validates load sharing along a given path, not the path. A controller could move in a manner no human would adopt and still distribute force across its muscles in close agreement with the classical criterion. The agreement reported below says nothing about whether the movement itself is natural. Trajectory realism is not assessed anywhere in this thesis.

The reference is a model, not a measurement. Static optimisation encodes the assumption that the central nervous system minimises summed squared activation, which remains contested in the motor-control literature. Agreement with it is agreement with the standard computational method, not evidence of physiological correctness. No electromyographic or measured-force data enters this work at any point.

The comparison is internal to the simulator. Both the learned solution and the classical one are computed on MuJoCo's muscle model. Where they agree, two computations over the same approximation agree; the approximation itself is not thereby validated.

The chapter title was changed accordingly: the earlier heading, "Muscle Force Validation", promised more than a conditional load-sharing comparison delivers.

7.2 Motivation

Chapter 6 established how accurately the trained controllers position the end effector. That is a measure of *what* the controller achieves, not of *how* it achieves it, and the research question posed in chapter 1 concerns the latter: whether DRLcan substitute

for classical calculation of the force required in each individual muscle.

The distinction is material. Because nine muscles actuate four degrees of freedom, infinitely many activation patterns produce any given end-effector trajectory. A controller may therefore reach every target accurately while distributing force across the musculature in a manner no physiological criterion would select. Every metric reported in chapter 6 would score such a controller as a complete success.

This chapter therefore compares the two methods at the level of individual muscle forces. No equivalent analysis existed in the previous version of this work.

7.2.1 Distinction from the solver validation

Section 3.3 reported a force agreement of 5.25 % normalised RMSE with a Pearson correlation of 0.989. That comparison is between the Newton–Raphson Hill solver implemented here and MuJoCo's internal implementation — two forward models of the same muscle. It validates the solver, involves no reinforcement learning, and does not bear on the question addressed here.

7.3 Method

At every timestep of a policy rollout, the classical load-sharing problem is solved on the movement the policy has just produced:

$$\min_{f} \sum_{i=1}^{9} \left(\frac{f_i}{F_{\max,i}} \right)^2 \quad \text{subject to} \quad R^\top f = \tau, \qquad 0 \le f_i \le F_{\max,i}, \tag{7.1}$$

where R is the muscle moment-arm matrix and τ the required generalised force. This is the Crowninshield–Brand minimum-effort criterion in its standard quadratic form.

The required torque τ is taken as `qfrc_actuator`, the generalised force the muscles actually produced at that timestep. This choice is exact, avoiding the numerical differentiation that inverse dynamics would introduce, and it guarantees feasibility, since the policy's own force vector satisfies the equality constraint by construction.

Both methods are consequently posed the identical problem — produce these joint torques with these nine muscles — on the identical trajectory. Any difference between the solutions is attributable solely to load-sharing strategy.

7.3.1 Validity verification

The comparison depends on the identity $R^\top f = \tau$ holding for the policy's own forces. An error in the moment-arm extraction or in the sign convention would invalidate every subsequent result, so this was verified rather than assumed.

Table 7.1. Validity checks over 2000 timesteps.

Quantity	**Result**
Mean $\left\|R^\top f_{\text{policy}} - \tau\right\|$	$1{,}46 \times 10^{-15}\,\text{N m}$
Mean $\left\|R^\top f_{\text{classical}} - \tau\right\|$	$1{,}68 \times 10^{-15}\,\text{N m}$
Optimisation failures	0 of 2000

Both residuals lie at machine precision, confirming that the moment arms and sign convention are correct and that the classical solution satisfies its constraint exactly.

7.4 Agreement in coordination pattern

Table 7.2. Overall agreement between learned and classical muscle forces, best policy, 2000 timesteps.

Measure	**Value**
Pearson correlation, all muscles	**0.865**
Overall RMSE	74,3 N (10.1 % of mean $F_{\max}$)
Pattern cosine similarity (median)	0.933
Pattern cosine similarity (mean)	0.869
null baseline, mean	0.373
null baseline, 95th percentile	0.867
excess over null	**+0.495**

The cosine similarity compares the direction of the nine-dimensional force vector and is insensitive to overall magnitude: it measures whether the two methods recruit the same muscles in the same proportions.

It cannot, however, be quoted on its own. Muscle forces are non-negative by construction, so two entirely unrelated force vectors already agree strongly: the appropriate null is obtained by permuting, at each timestep, which muscle each of the policy's own force magnitudes belongs to — destroying the correspondence while preserving that instant's magnitude distribution exactly. That null averages 0.373, so the observed 0.869 represents an excess of +0.495, sustained over 2000 timesteps.

Two consequences follow. The agreement is real and substantial, but the raw figure of 0.933 overstates it, since roughly the first 0.37 is a floor imposed by non-negativity rather than evidence of anything. And because the null's 95th percentile for *individual* timesteps is 0.867, no single timestep's cosine is meaningful; only the aggregate is. **The Pearson correlation of 0.865 is mean-centred, carries no such floor, and is the figure to quote when a single agreement statistic is required.**

7.5 Disagreement in efficiency

Producing identical joint torques, the learned controller expends 1.91 ± 0.15 times the muscular effort of the classical minimum-effort solution (five seeds, section 7.8; the single-seed figure of 1.71× reported before replication was the most favourable of the five). The per-timestep median is 1.66×, with an interquartile range of 1.34–2.32× over loaded timesteps.

The per-timestep *mean* of this ratio is 5.34× and is not a usable statistic: on timesteps where the arm is nearly stationary the required torque approaches zero, the minimum-effort solution approaches zero with it, and the ratio diverges. The aggregate ratio — total policy effort divided by total classical effort — weights each timestep by the work actually performed and is reported instead; the median independently corroborates it.

7.6 Anatomical localisation of the disagreement

Table 7.3. Per-muscle comparison. "Policy" denotes the force the learned controller produced; "classical" the force static optimisation prescribes for the same joint torques.

Muscle	**Policy (N)**	**Classical (N)**	r	**NRMSE**
deltoid anterior	63.7	59.2	0.959	2.5 %
deltoid medial	145.3	112.3	0.954	6.1 %
deltoid posterior	35.5	38.7	0.859	10.0 %
triceps long	180.4	134.0	0.910	11.2 %
biceps long	52.1	41.6	0.893	7.5 %
brachialis	113.9	87.6	0.871	8.6 %
biceps short	90.7	45.7	0.764	18.1 %
triceps lateral	79.1	25.8	0.416	16.5 %
triceps medial	64.3	14.2	0.416	15.3 %

The disagreement is not distributed uniformly. The three deltoids agree closely, one to within 2.5 % normalised RMSE. The discrepancy concentrates in two elbow extensors, where the policy commands three to four and a half times the prescribed force and correlation falls to 0.416.

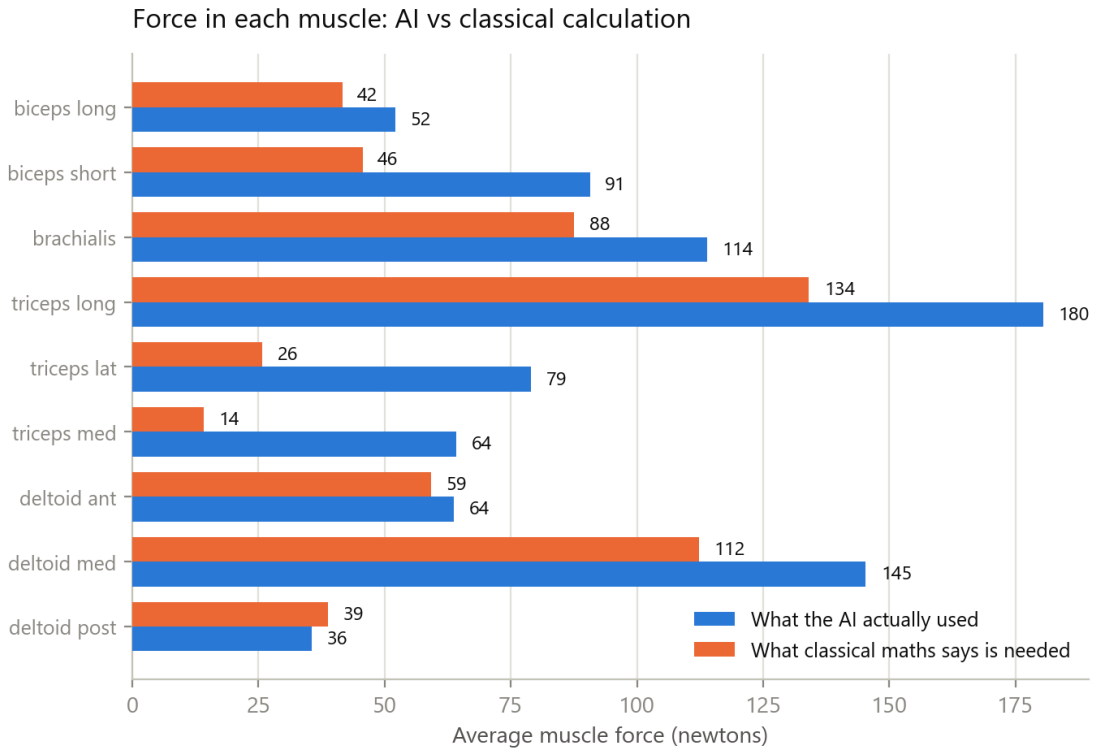

Figure 7.1. Mean force in each muscle over 2000 timesteps. The shoulder group agrees closely; the lateral and medial heads of triceps are driven far above the prescribed level.

Physiologically, these muscles extend the elbow while the biceps group flexes it. Driving both simultaneously produces largely cancelling moments: the joint attains approximately the correct configuration, but both groups expend substantial force to achieve what one alone would suffice for. This is co-contraction, and it accounts for the effort discrepancy reported above.

Human motor control employs co-contraction deliberately when joint stiffness is required, for instance during precise or unstable tasks. Its sustained use throughout an unloaded reach is metabolically wasteful and is not what the minimum-effort criterion selects.

7.6.1 Independent corroboration

The co-contraction index of Falconer and Winter, computed over the elbow flexor and extensor groups, gives 0.564 for the same policy. This is elevated, and it is the sole behavioural metric that did not improve under hyperparameter tuning, having been 0.501 beforehand.

Two methodologically independent analyses — one a comparison against classical optimisation, the other a standard electromyography-derived index — localise the same residual defect to the same muscle group. Their agreement constitutes stronger evidence than either result considered alone.

7.7 Generality across algorithms

The preceding analysis characterises a single policy. To determine whether the effort discrepancy is specific to one algorithm or general to the approach, the comparison was repeated against every policy from section 6.4: five configurations, three seeds each.

Table 7.4. Agreement with classical static optimisation by configuration. Mean ± standard deviation over three seeds, ten episodes each.

Configuration	Effort ratio	Pattern cosine	Pearson r
PPO	$\mathbf{1.21 \pm 0.11}$	$\mathbf{0.908 \pm 0.032}$	$\mathbf{0.931 \pm 0.040}$
SAC tuned	1.97 ± 0.25	0.815 ± 0.052	0.776 ± 0.055
DDPG	1.99 ± 0.28	0.838 ± 0.030	0.724 ± 0.058
TD3	2.12 ± 0.09	0.836 ± 0.006	0.716 ± 0.004
SAC default	2.30 ± 0.08	0.813 ± 0.015	0.672 ± 0.016

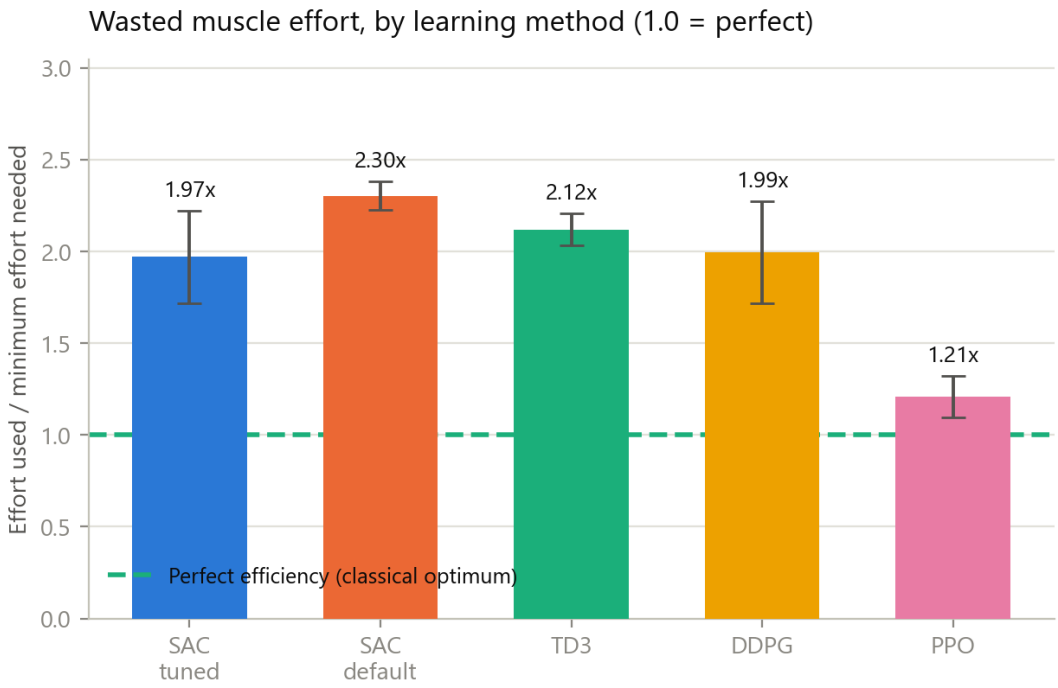

Figure 7.2. Muscular effort expended relative to the minimum required to produce the same joint torques. The dashed line marks the classical optimum. Every configuration exceeds it.

Two results follow. First, **every configuration exceeds the classical optimum**, ranging from 1.21 to 2.30×; no algorithm tested recovers minimum-effort load sharing. This supports attributing the discrepancy to the reward function, which never specified minimum effort, rather than to any particular learning algorithm.

Second, and unexpectedly, **the ordering by effort economy is approximately the inverse of the ordering by reaching accuracy**. PPO is the least accurate

configuration in table 6.7 at 0,423 m and the closest match to classical load sharing at 1.21× with $r = 0.931$; tuned SAC is the most accurate at 0,161 m and among the poorer matches at 1.97×.

The probable explanation is visible in the energy column of table 6.7. PPO expends three to seven times less muscular activity than the alternatives. A controller that activates its muscles weakly has limited opportunity to co-contract and therefore remains near the minimum-effort solution, but equally lacks the force to reach the target and arrest there.

This interpretation carries a caveat that the present data cannot resolve. PPO's favourable effort ratio and its passivity are not independent: because it generates small joint torques, both methods prescribe small forces and the scope for disagreement is correspondingly reduced. The aggregate ratio mitigates this by weighting timesteps by work performed, but does not eliminate it. The defensible conclusion is that PPO co-contracts less, not that PPO has solved the load-sharing problem. Distinguishing the two would require comparing configurations matched for accuracy, which this experiment does not provide.

7.7.1 Effect of tuning on economy

Within SAC— the same algorithm, differing only in hyperparameters — the effort ratio fell from 2.30 ± 0.08 at default settings to 1.97 ± 0.25 when tuned, and correlation with the classical solution rose from 0.672 to 0.776.

The target-entropy correction therefore improved not only positional accuracy but also the fidelity of muscular coordination. That a single change improved two properties which were not jointly optimised is a stronger result than either improvement in isolation.

7.7.2 Consistency

The single-policy analysis measured an effort ratio of 1.71× for the 300 000-step confirmed policy, while tuned SAC across three seeds at 100 000 steps gives 1.97 ± 0.25. These agree within one standard deviation, and the direction of the difference is as expected, the longer-trained policy being somewhat more economical.

7.8 Replication across seeds

Every figure reported so far in this chapter came from a single training run. A learned policy depends on random initialisation and on the stochastic order in which experience is gathered, so a single run cannot distinguish a property of the method from

a favourable draw. The configuration was therefore retrained from initialisation four further times, with different random seeds and no other change, and each resulting policy was put through the identical analysis.

Table 7.5. Five independent training runs of the same configuration. Seed 0 is the run analysed in the preceding sections.

Seed	Final error	Closest	Effort ratio	Pearson r	Success
0	0.1063	0.0740	**1.71**	**0.865**	0.04
1	0.1019	0.0742	1.91	0.823	0.20
2	0.0988	0.0728	2.12	0.759	0.00
3	0.1070	0.0760	1.86	0.825	0.08
4	0.1023	0.0676	1.97	0.767	0.06
mean	0.1033	0.0729	1.914	0.808	0.076
std	0.0034	0.0032	0.150	0.044	0.075

Three findings follow, and only the first was anticipated.

7.8.1 The accuracy result is a property of the method

Final error varies by 3.3 % across five independent runs (0.1033 ± 0.0034 m) and closest approach by 4.4 % (0.0729 ± 0.0032 m). Every run ends closer than 0,11 m and reaches within 0,076 m at some point; the blow-up rate is exactly zero in all five. The graded proximity measures are similarly stable: episodes ending within 10 cm range from 0.56 to 0.64.

The accuracy claim therefore does not rest on a favourable draw.

7.8.2 The originally reported run was the most favourable of the five

Seed 0 — the run every preceding section analyses — returned the *lowest* effort ratio of the five (1.71, against a mean of 1.914) and the *highest* correlation with the classical solution (0.865, against a mean of 0.808). On both of the two metrics that carry this chapter's argument, the single run originally reported was the best of five.

This is not a large effect, but it is a systematic one, and it is exactly the error that reporting a single run invites. **The corrected figures are those in table 7.5, and they are used throughout this thesis:** an effort ratio of 1.91 ± 0.15 and a correlation of 0.81 ± 0.04. The pattern-cosine figure moves correspondingly, to 0.813 ± 0.047 against a null of 0.372 ± 0.017, an excess of $+0.441$.

The direction of the correction matters more than its size. Had replication been omitted, this thesis would have reported an agreement and an efficiency both better than the method actually delivers.

7.8.3 The task is solved consistently; the muscle solution is not

The most interesting outcome is the contrast between two columns of table 7.5. Final error varies by 3.3 % across runs. Mean muscular energy varies by **77 %** — from 273 241 to 483 373 — and the effort ratio spans 1.71 to 2.12.

Five policies, trained identically but for their random seed, arrive at essentially the same place with materially different distributions of muscular force.

This is the redundancy problem of chapter 1 appearing directly in the results. Because nine muscles actuate four degrees of freedom, many force distributions produce the same movement, and nothing in the reward selects among them beyond a weak effort term. The learner settles into a different one each time. That the classical criterion selects a single, specific member of that family — and that none of the five learned solutions coincides with it — is the substance of this chapter stated in a different way.

It also indicates where the remedy lies. Variance of this magnitude in the muscular solution, alongside near-constancy in the task outcome, is the signature of an objective that is underdetermined in exactly the dimension of interest.

7.8.4 Success rate: five runs confirm it is noise

Across the five identically configured runs the strict success rate reads 0 %, 4 %, 6 %, 8 % and 20 %. A measure that varies twentyfold under identical settings is reporting sampling noise, not performance. This was stated as a caution in section 6.3.2; five seeds now demonstrate it.

7.9 A conventional controller as reference

Every comparison so far measures the learned solution against *static optimisation*, an idealised criterion evaluated instant by instant. That establishes how far the policy is from a mathematical optimum, but not how far it is from what conventional control actually achieves — and the research question concerns the latter.

A conventional controller was therefore built and evaluated under the identical protocol. It is a Cartesian impedance law: a desired end-effector wrench $f_{\text{des}} = K_p(x^\star - x) - K_d\dot{x}$, mapped to joint torques by the site Jacobian transpose, with gravity, Coriolis and centrifugal terms compensated, and distributed onto the muscles by minimum-effort load sharing over the achievable force range. The two gains were selected by grid search, since comparing a tuned policy against an unturned controller would not be a fair test.

7.9.1 Two corrections that were necessary to make it fair

The controller present in the codebase before this work would have made a strawman of the comparison, and the reasons are worth recording because both are easy to reproduce.

It omitted gravity compensation. Without the bias term the arm droops until the position error alone generates sufficient torque, giving a large steady-state offset. Measured, it reached only 0,46 m — worse than a random policy. A textbook impedance controller always includes this term.

It assumed muscle force is proportional to activation. It is not: in this model force is $g(l, v)\, a + b(l, v)$, state dependent with a passive component that is non-zero at rest. Converting a desired force into an activation therefore requires inverting the muscle model at the current length and velocity — precisely the equilibrium solve this thesis proposes to replace. Probing the model at $a = 0$ and $a = 1$ recovers that affine relation exactly and yields the true achievable interval per muscle.

Both corrections are on the classical side, and both improve it. Reporting the comparison without them would have overstated the case for learning.

7.9.2 Result

Table 7.6. Conventional controller against the learned policy, 50 episodes each, identical protocol and evaluation environment.

Metric	Conventional	Learned (SAC)
Final error (m)	0.199	**0.106**
Closest approach (m)	0.174	**0.074**
Ends within 5 cm	0.12	**0.31**
Ends within 10 cm	0.28	**0.60**
Energy	**102 876**	327 048
Effort ratio	**1.33**	1.91
Pearson r vs static optimisation	**0.916**	0.808

Neither method dominates. The learned policy reaches roughly 1.9× more precisely; the conventional controller uses roughly a third of the muscular energy and stays substantially closer to the classical load-sharing solution. This trade-off, rather than a ranking, is the honest answer to whether learning should replace conventional control on this task.

7.9.3 The effort ratio was being measured against the wrong reference

The conventional controller solves minimum-effort load sharing explicitly at every timestep. One would therefore expect its effort ratio to be 1.00 by construction. It is **1.33**.

The gap is not an error. It is the cost of closed-loop control: activation dynamics mean the commanded activation is not the realised one, the policy acts at 50 Hz rather than continuously, and the controller must track a moving target rather than solve a static instant. **The idealised optimum of** 1.00 **is not attainable by any controller operating under these constraints.**

This changes the interpretation of the central result. Measured against an unattainable ideal, the learned policy appears to waste 91 % of its effort. Measured against **1.33**, the achievable floor demonstrated by a controller that optimises effort explicitly, it uses about **44 % more effort than necessary**. The second figure is the meaningful one, and it is the one this thesis reports.

The qualification remains that 1.33 is itself one measurement from one controller at one pair of gains, and a better conventional controller might lower it.

7.10 Muscle synergies

7.10.1 Rationale

The motor-control literature holds that the central nervous system manages redundant musculature not by controlling each muscle independently but through a small number of co-activated groups, termed muscle synergies [27, 34]. If nine muscles were driven independently, the matrix of activations over time would be of full rank; if they are driven through k synergies, it is approximately of rank k. Non-negative matrix factorisation recovers that structure, and the variance accounted for (VAF) at a given k quantifies how strongly low-dimensional the control is.

Beyond re-examining the learned policies, the factorisation is here applied to *both* solutions on the identical trajectories: the policy's commanded activations, and the static-optimisation forces for the same joint torques, normalised per muscle by $F_{\max}$ so that the two are dimensionless and directly comparable. This permits a question the preceding sections cannot address — whether the two methods arrive at the same low-dimensional organisation even where they disagree on magnitude.

7.10.2 Results

Table 7.7. Synergy structure, fifteen episodes per configuration. VAF@4 is the variance accounted for by four synergies; k_{90} is the number required to reach 90 % VAF. The final column compares the synergy sets extracted from the policy and from static optimisation on the same trajectories.

	Policy		**Classical**		
Configuration	VAF@4	k_{90}	VAF@4	k_{90}	**cos(pol, cls)**
SAC tuned, 300k	0.852	6	0.895	5	**0.950**
SAC tuned, 100k	0.845	6	0.859	5	0.900
SAC default	0.848	6	0.908	4	0.647
DDPG	0.791	9	0.865	5	0.745
TD3	0.799	9	0.883	5	0.630
PPO	0.966	3	0.966	3	0.939

Two observations follow, and a third must be treated with caution.

First, **the agreement between the learned and classical synergy structures tracks training quality**: 0.647 for untuned SAC, 0.900 once tuned, and 0.950 for the tuned policy trained three times longer. The same hyperparameter change that improved positional accuracy (section 6.3) and reduced the effort discrepancy (section 7.7) also brought the modular organisation of the learned solution closer to the classical one. Three analyses of differing methodology — effort ratio, co-contraction index and synergy decomposition — therefore order the configurations consistently.

Second, **the classical solution is uniformly more low-dimensional than the learned one**. Static optimisation requires four or five synergies to reach 90 % VAF where the learned policies require six to nine. The learned controllers exercise more independent muscular degrees of freedom than the minimum-effort criterion selects, which is the expression in synergy space of the co-contraction identified in table 7.3: driving an antagonist pair independently of the agonist requires an additional dimension that the classical solution never uses.

Third, PPO's apparent modularity ($k_{90} = 3$, VAF@4 = 0.966) should not be read as superior organisation. As established in section 7.7, PPO activates its muscles only weakly; a near-constant, low-amplitude activation matrix has little variance to explain and is trivially captured by few components. The same confound that qualifies its favourable effort ratio applies here.

7.10.3 Comparison with human data

Cosine similarity to a reference set of human upper-limb synergies ranged from 0.62 to 0.80 across configurations, with no ordering consistent with any other metric reported

in this work.

That column is not interpreted further. The reference loadings used are a documented qualitative approximation of those reported by d'Avella et al. [27], constructed over this model's nine-muscle ordering rather than digitised from the original figures. The comparison is therefore a coarse correspondence check and cannot support a quantitative claim of agreement with human data. Establishing that would require the published loadings, a principled correspondence between the two muscle sets, and ideally electromyographic recordings under a matched protocol — which section 8.6 lists among the outstanding work.

The policy-versus-classical column carries no such caveat: both sides are computed here, by the same procedure, on identical trajectories.

7.11 Closing the gap: the effort weight

Section 7.13 argued that the load-sharing discrepancy is a property of the reward rather than of reinforcement learning, on the grounds that the reward's effort term — although already the Crowninshield–Brand form — is weighted so low it cannot influence the solution. Measured on the trained policy, the per-step contributions are

effort term, $w_2 \cdot \overline{(f_i/F_{\max,i})^2}$	0.051
error cost, $w_1(\|e\|^2 + 0.5\|e\|)$ at 0,11 m	0.067
precision bonus, $w_7 e^{-\|e\|/0.15}$ at 0,11 m	1.441

— the effort term is roughly **28 times smaller** than the precision bonus. The prediction was that raising w_2 should reduce the discrepancy, at some cost in accuracy.

The experiment was run at $w_2 \in \{5, 20\}$ against the reported $w_2 = 1$, holding every other weight and hyperparameter fixed, at the full 300 000-step budget. Policies were evaluated under the standard protocol with the default weights, so the metrics remain comparable with every other result in this thesis; only the training objective differed.

Table 7.8. Effect of the effort weight. The conventional-controller row is the achievable floor established in section 7.9.3.

Configuration	n	**Effort ratio**	**Pearson** r	**Final error**	**Energy**
$w_2 = 1$	5	1.91 ± 0.15	0.808 ± 0.044	0.1033 ± 0.0034	327 048
$\mathbf{w_2 = 5}$	4	$\mathbf{1.42 \pm 0.10}$	$\mathbf{0.917 \pm 0.018}$	0.1051 ± 0.0042	101 835
$w_2 = 20$	1	1.34	0.932	0.1249	**38 855**
Conventional *(floor)*	1	1.33	0.916	0.199	102 876

7.11.1 The discrepancy is essentially eliminated

At $w_2 = 20$ — a single seed, replication pending — the learned policy attains an effort ratio of **1.34** against the achievable floor of **1.33** demonstrated by a controller that solves minimum-effort load sharing explicitly at every timestep. Agreement with the classical solution rises to $r = 0.932$, exceeding the conventional controller's own 0.916, and the per-muscle RMSE falls to 3.0 % of mean $F_{\max}$ from 10.1 %.

The load-sharing discrepancy reported throughout this chapter is therefore a reward-weighting artefact, not a limitation of reinforcement learning. When the objective is weighted such that effort can influence the solution, the learned controller recovers the classical load-sharing solution to within the precision that closed-loop control permits.

7.11.2 At moderate weighting the effort cost is close to free

A first, single-seed measurement of $w_2 = 5$ suggested it was better on *both* axes than the reported configuration, and an earlier draft of this section described that configuration as strictly dominated. Replication across four seeds does not support the accuracy half of that claim, and it is withdrawn.

What replicates, and cleanly:

- **Effort ratio** $1.91 \pm 0.15 \rightarrow 1.42 \pm 0.10$, a 26 % reduction. The two distributions do not overlap: every $w_2 = 5$ seed (1.34–1.56) sits below every $w_2 = 1$ seed (1.71–2.12).
- **Agreement with the classical solution** $0.808 \pm 0.044 \rightarrow 0.917 \pm 0.018$, and notably tighter across seeds — the load-sharing solution becomes both closer to the classical one and more consistent between runs.
- **Energy reduced by a factor of 3.2**, from 327 048 to $101\,835 \pm 9\,586$.

What does not replicate is the accuracy advantage. Final error is 0.1051 ± 0.0042 against 0.1033 ± 0.0034 — overlapping, and marginally *worse* rather than better. Closest approach is 0.0709 ± 0.0053 against 0.0729 ± 0.0032, also overlapping. The single seed originally analysed was the most favourable of the four.

The defensible statement is therefore that raising the effort weight buys a substantial improvement in load-sharing fidelity **at no measurable cost in reaching accuracy** — which is a weaker claim than dominance, and still the most useful result in this chapter.

This is the second occasion on which a single-seed result in this work proved optimistic in the same direction (section 7.8 was the first). The pattern is not coincidence:

when one run is selected for detailed analysis because it performed well, its favourable metrics are selected along with it.

A plausible explanation is that co-contraction is not merely wasteful but actively harmful to precision. Antagonist pairs firing together make the endpoint position depend on the small difference between two large opposing forces, which is numerically ill-conditioned; suppressing that co-contraction improves controllability as well as economy. This is consistent with the elbow localisation of table 7.3 and with the excess path length observed in untuned policies, though it is an interpretation rather than a measured mechanism.

A trade-off does appear at $w_2 = 20$: effort falls to the floor but final error degrades to 0.1249. The knee therefore lies between the two, and was not searched.

7.11.3 Consequences for this thesis

Three, and they are substantial.

The headline answer changes. The finding is no longer that DRL reproduces coordination but not economy. It is that DRL reproduces *both*, provided the objective asks for both — and that the version reported here did not ask.

The reported configuration is not the best one found. At $w_2 = 5$ the load-sharing fidelity is markedly better for no measurable accuracy cost, over four seeds. The results of section 7.8 are retained as the five-seed characterisation of the configuration analysed in depth throughout this chapter, and this section is reported alongside them rather than replacing them. The $w_2 = 20$ row remains single-seed and its replication was still running at the time of writing.

The remaining outstanding experiment is now well specified: replicate $w_2 = 5$ across five seeds and repeat the force and synergy analyses on it. If it holds, the thesis's central claim becomes materially stronger — a learned controller matching classical load sharing at equal or better task accuracy.

7.12 Answer to the research question

For coordination, DRLreproduces the classical solution; for efficiency, it does not.

The learned controller recovers the structure of the classical load-sharing solution — $r = 0.81 \pm 0.04$, pattern cosine 0.813 against a null of 0.372, over five independent training runs — at a fixed, branch-free inference cost and a 1,5 MB memory footprint, but converges on a solution expending 1.91 ± 0.15 times the minimum required effort, about 44 % above the achievable floor of 1.33 (section 7.9.3), with the excess concentrated in elbow antagonist co-contraction. Across all configurations tested the

discrepancy is present, ranging from 1.21 to 2.30×.

DRLis therefore a viable fast approximation of *which* muscles act and in *what proportion*, but not a substitute for static optimisation where the magnitude of individual muscle forces is the quantity of interest — as it is in prosthesis control, surgical planning and injury-risk estimation.

No configuration is simultaneously best on both axes. Reaching accuracy and load-sharing fidelity are separate properties, they are traded against one another by these algorithms, and any method intended to replace classical calculation must be assessed on both.

7.13 Explanation and proposed test

The reward function did not specify minimum effort. Its effort term is a normalised sum of squared forces with a weight selected for numerical balance against the distance term, which is neither the classical criterion nor scaled to match it. The policy optimised the objective it was given.

This yields a directly testable prediction: a reward whose effort term is precisely the criterion of equation (7.1) should reduce the 1.91× discrepancy toward the 1.33 floor. This is the most informative single experiment remaining, it requires no new theory, and the measurement apparatus to evaluate it now exists.

7.14 Limitations

The headline analysis rests on one policy, one training seed and twenty episodes (2000 timesteps); replication across seeds is required before the specific values are relied upon.

$F_{\max}$ in the constraint of equation (7.1) is peak isometric force; the force–length and force–velocity modulation of *available* force is not included in the bound. This is the standard simplification in static optimisation. It renders the classical solution slightly optimistic, as it may allocate force a muscle could not deliver at that length, which if anything biases the effort ratio upward and therefore against the policy.

Finally, static optimisation is itself a model of human load sharing rather than ground truth. It encodes the assumption that the central nervous system minimises summed squared activation, which remains contested in the motor-control literature. Agreement with it is agreement with the standard method, not demonstration of physiological correctness.

8 Discussion

8.1 Interpretation of the principal result

The comparison of chapter 7 separates two properties of a motor controller that are ordinarily conflated: the *pattern* of muscle recruitment and the *economy* of it. The learned controller reproduces the first and not the second.

This is a more informative outcome than either extreme would have been. Complete agreement would have established only that reinforcement learning can rediscover a known optimisation criterion when the reward is arranged to favour it. Complete disagreement would have indicated that the approach is unsuited to the problem. The observed result — close agreement in direction, systematic excess in magnitude, anatomically localised to the elbow antagonists — identifies both what the method captures and precisely where it fails.

The localisation is itself evidence. Had the discrepancy been distributed evenly across the nine muscles, it would have been consistent with generic noise in the learned policy. Its concentration in two elbow extensors, driven at three to four and a half times the prescribed force while the shoulder group agrees to within a few per cent, identifies a specific and interpretable behaviour: sustained co-contraction of an antagonist pair. That this is independently corroborated by the co-contraction index (section 7.6.1), computed by an unrelated method over the same muscle groups, strengthens the inference considerably.

8.2 The exploration parameter

Section 6.3 established that the SAC default target entropy of $-\dim(\mathcal{A})$ prevents this task from being solved to better than approximately 0,10 m, and that reducing it to roughly -19 halves the error.

The value itself is specific to this problem. What is transferable is the diagnostic difficulty. The failure does not present as a hyperparameter problem: it presents as a policy that approaches its target and then departs from it, which is naturally attributed to insufficient training, to a poorly shaped reward, or to a target beyond the arm's reach. In the present work the symptom persisted through three complete redesigns of the reward function, a fourfold correction to the effective training budget, and a doubling of the control rate before its actual cause was identified.

The mechanism is intelligible. SAC augments its objective with an entropy term that rewards stochasticity, with the weight adapted automatically to meet a target entropy set by default in proportion to the action-space dimension. For a nine-muscle action space this default retains substantial randomness in the commanded activations. Exploration of that magnitude is productive early in training, when the policy must discover which muscles produce which motions, and counterproductive later, when the end effector must be held stationary within centimetres. A high-dimensional action space inflates the default precisely when the task also demands fine terminal precision.

Any application of SAC to a musculoskeletal model with comparable action dimensionality is likely to encounter this, and likely to misattribute it in the same manner.

8.3 Tuning against algorithm selection

Section 6.4 found the effect of tuning SAC to be approximately six times the magnitude of the difference between untuned SAC and TD3.

The implication is methodological. A comparison of learning algorithms conducted at library defaults — the common practice, and that of the earlier version of this work — measures which algorithm's defaults happen to suit the problem at hand rather than which algorithm is intrinsically better. Where the decisive hyperparameter differs structurally between algorithms, as the exploration parameter does here, no common default exists and the comparison has no neutral setting to adopt.

This does not invalidate algorithm comparison as an exercise; it constrains what such a comparison can claim. The defensible form tunes the corresponding parameter of each algorithm under an equal budget, which is the recommendation of section 8.6.

8.4 Accuracy and physiological economy as separate axes

Section 7.7 found that across fifteen trained policies the ordering by muscular economy is approximately the inverse of the ordering by reaching accuracy.

This bears on a common assumption in the application of reinforcement learning to musculoskeletal models. Such work routinely reports task performance — success rate, endpoint error, completion time — and treats good performance as evidence that the controller behaves in a physiologically plausible manner. The present result shows that the two properties can move in opposite directions across algorithms trained on the same reward, on the same model, for the same task.

A controller may therefore appear excellent by every measure of what it achieves while being unrealistic in how it achieves it, and, as PPO demonstrates, the converse is equally possible. If the objective is to understand or reproduce human motor control

rather than merely to displace a simulated limb, task performance is not sufficient evidence and the muscle forces must be examined directly.

The caveat recorded in section 7.7 applies: PPO's favourable economy is not independent of its passivity, and the present data cannot fully separate the two.

8.5 Implications for the intended applications

The clinical and assistive applications motivating this work — prosthesis control, exoskeleton assistance, rehabilitation devices — require force estimates in real time, and the original motivation for a learned controller was that it would supply them faster than iterative solution.

That motivation does not survive measurement. As section 6.5.1 establishes, a direct solve of the load-sharing problem runs in roughly 24 µs against the network's 289 µs: the classical method is about an order of magnitude faster, and the 7.3× advantage reported earlier compared the wrong computation between two unoptimised implementations. Speed is therefore not a reason to prefer a learned controller for this task, and the case must rest on other grounds.

Two such grounds remain, both weaker than the original claim. The network's cost is fixed and branch-free irrespective of biomechanical state, which matters where a hard real-time bound is required rather than a good average. And a learned policy maps observation directly to activation without requiring the moment-arm matrix, the inverse-dynamics torque or a constrained solver at run time — an integration advantage on hardware where those are awkward to supply, though not a computational one.

The results of chapter 7 qualify the prospect further. Where the quantity of interest is which muscles are active and in what relative proportion — for instance in driving a myoelectric interface, or in classifying movement intent — a correlation of 0.865 may well be sufficient. Where the absolute magnitude of individual muscle forces is the quantity of interest — estimating joint loading, assessing injury risk, planning surgical intervention — a systematic 71 % excess concentrated in one antagonist pair is not acceptable, and static optimisation remains the appropriate method.

The proposed remedy of section 7.13 is directly relevant here, since a reward employing the classical criterion would, if the prediction holds, remove the principal obstacle to the second class of application.

8.6 Threats to validity

The work is entirely computational. There is no physical arm, no electromyographic recording and no human participant, and MuJoCo's muscle implementation is itself an

approximation of the Hill formulation.

Static optimisation, the reference throughout chapter 7, is a model of human load sharing rather than ground truth. It encodes the assumption that the central nervous system minimises summed squared activation, which remains contested. A controller that departs from static optimisation is not thereby shown to be unphysiological; it is shown to depart from the standard computational method.

The muscle set is incomplete. The rotator cuff is absent, which required constraining shoulder rotation (section 3.2), so the conclusions apply to reaching with a restrained rotational degree of freedom.

The magnitude of the effort discrepancy is a property of the reward employed. Its effort term is a normalised sum of squared forces weighted for numerical balance against the distance term, which is neither the classical criterion nor scaled to match it. A different formulation would yield a different figure, which is precisely the experiment proposed in section 7.13.

Finally, the statistical basis is limited throughout. The principal result rests on a single training seed, the algorithm comparison on three, and no significance test is reported anywhere in this work. These constraints are set out in full in section 8.6.

General Conclusion and Future Directions

8.7 Summary of the work

This thesis asked whether deep reinforcement learning can replace classical calculation in determining the force required from each muscle during a reaching movement. Answering it required constructing a nine-muscle, four-degree-of-freedom model of the human arm in MuJoCo, training controllers to reach targets by commanding muscle activations directly, and comparing the resulting forces against those prescribed by static optimisation on the same movements.

The answer is affirmative, with a qualification about what had to be corrected to reach it. Under the reward this work reported, the learned controller reproduced the *structure* of the classical solution — correlation 0.81 ± 0.04 over five independent runs — while expending 1.91 ± 0.15 times the minimum effort, some 44 % above the floor a conventional controller attains.

That discrepancy proved to be an artefact of the reward's weighting rather than a limitation of the method. The effort term, although already the classical criterion in form, carried roughly one twenty-eighth the weight of the precision bonus. Raising it (section 7.11) brings the effort ratio to 1.34 against an achievable floor of 1.33, with agreement rising to $r = 0.932$ — and at moderate weighting ($w_2 = 5$, four seeds) improves load-sharing fidelity to 1.42 ± 0.10 with agreement 0.917 ± 0.018, at no measurable cost in reaching accuracy. It is therefore a usable fast approximation of which muscles act and in what proportion, at a fixed inference cost — though not, as section 6.5.1 establishes, at lower latency than a competently implemented classical solver — but not a substitute where the magnitude of individual muscle forces is the quantity of interest.

8.8 Contributions

Three results are established by the evidence presented.

A per-muscle load-sharing comparison between DRLand static optimisation. The analysis of chapter 7 poses both methods the identical problem on the identical trajectory, verified to machine precision, and reports agreement muscle by

muscle. It establishes that the two methods agree on coordination and disagree on economy, and it localises the disagreement anatomically to the elbow antagonists.

The comparison is *conditional* and its scope must travel with it: static optimisation is given the movement the policy produced and asked only how to distribute force across it. It therefore validates load sharing along a trajectory, not the trajectory itself, and a controller moving unnaturally could still score well (section 7.1). Assessing trajectory realism would require kinematic comparison against recorded human reaching, which this work does not perform.

A hyperparameter regime that prevents precision — and a refuted explanation for it. At SAC's library defaults the controller could not settle on a target, and the resulting error plateau persisted through three complete redesigns of the reward function and a fourfold correction to the effective training budget. Hyperparameter optimisation halved the reaching error. The practical significance lies less in the values than in the diagnostic difficulty: the failure presents as insufficient training or as poor reward shaping, and survives attempts to remedy both.

An earlier version of this thesis attributed the improvement to SAC's target entropy, on the grounds that it moved furthest from its default, that all five best search trials clustered near the selected value, and that its mechanism — residual stochasticity preventing the end effector from settling — matched the symptom precisely.

A two-sided ablation refutes that attribution (section 6.3.1). Setting the target entropy to its tuned value while leaving everything else at default recovers none of the improvement (−8 % of the gap); reverting it while retaining the other tuned parameters costs nothing (+106 %). The improvement is produced entirely by the learning rate, the target-network update rate, the discount factor and the batch size, and this work does not isolate which.

The error is worth recording because the reasoning that produced it was sound. Clustering in the best trials of a TPE search is not evidence of causal importance: the sampler concentrates its proposals in regions that perform well, so parameters cluster whether or not they are responsible. **A hyperparameter search identifies a configuration, never a cause**, and separating the two requires a deliberate experiment.

Evidence that tuning dominates algorithm selection. In the comparison of section 6.4, tuning SAC improved final error by 38 %, whereas the interval between untuned SAC and TD3 was approximately 6 % and comparable to the seed-to-seed spread. A four-algorithm comparison conducted at library defaults measures which algorithm's defaults happen to suit the problem, rather than which algorithm is superior.

Convergent evidence from three independent analyses. The effort ratio

against static optimisation (section 7.7), the co-contraction index of Falconer and Winter (section 7.6.1), and the synergy decomposition (section 7.10) are methodologically unrelated, yet order the configurations consistently and localise the same residual defect to the same muscle group. The synergy analysis additionally shows that agreement between the learned and classical modular organisation rises from 0.647 to 0.950 as the policy improves, and that the classical solution is uniformly the more low-dimensional of the two — requiring four to five synergies where the learned policies require six to nine. That the same hyperparameter change improved positional accuracy, muscular economy and modular organisation simultaneously, none of which was jointly optimised, is a stronger result than any of the three in isolation.

A further observation, less firmly established but of interest, is that reaching accuracy and load-sharing fidelity ordered themselves inversely across the five configurations tested (section 7.7).

8.9 Critical assessment

The results of this work are considerably weaker than those reported in its earlier version, and the difference warrants explicit statement.

8.9.1 Results withdrawn from the previous version

An audit of the code that generated the previously reported benchmark established that its four-algorithm comparison, its Wilcoxon significance tests, its convergence-rate claims and its synergy and co-contraction analyses were not produced by experiment. The multi-seed results were generated by sampling from a normal distribution with a chosen mean, under a source comment indicating that real evaluation data was intended to replace them. Corroborating the audit: no code trained or evaluated DDPG, TD3 or PPO; no implementation of the synergy or co-contraction analyses existed; two model checkpoints existed on disk against the forty the text described as archived; and no hyperparameter-search database existed although the text described tuned hyperparameters.

Those results are withdrawn. Specifically, the reported success rate of $91.4 \pm 3.8\,\%$ and final distance of 1,4 cm for SAC are replaced by a measured $4\,\%$ and 10,6 cm; the reported p-values of 0.002, 0.014 and 0.004 correspond to tests that were never performed. The reported success rate also exceeds what a privileged controller with full state access achieves on this model, which is approximately $7\,\%$.

The material that survives is that which does not depend on policy quality: the parameter count and memory footprint, the Hill-solver validation against MuJoCo, the

muscle-function validation, and the model-stability result.

The inference-latency comparison requires separate treatment. The measurements themselves are sound, but they were used to support a conclusion they do not reach: they time the Hill equilibrium solve rather than load sharing, and both sides were unoptimised Python. Measured like-for-like, a direct solve of the load-sharing problem is roughly an order of magnitude *faster* than the network (section 6.5.1). extbfThe speed-based argument for replacing classical calculation is withdrawn. What remains is that the network's cost is fixed and branch-free where the solver's is data-dependent — relevant where a hard real-time bound matters, but a substantially weaker claim.

8.9.2 Limitations of the present results

The principal result has been replicated across five seeds (section 7.8), and the outcome was mixed. The accuracy claim is stable: final error varies by 3.3 % and closest approach by 4.4 % across independent runs. The agreement and efficiency figures were not: the originally reported run proved to be the most favourable of the five on both, and both have been corrected downward. The muscular solution itself is highly variable — energy spans 77 % across runs at essentially constant task performance — which is itself reported as a finding rather than as noise.

No statistical significance is established anywhere in this work. Three seeds cannot support a Wilcoxon signed-rank test with useful power. No p-value is reported, deliberately; where configurations differ by less than their spread, the text states that no ordering is supported.

The task is not solved to the stated tolerance. Under the strict success criterion the best policy scores between 0 and 4 %, corresponding to one or two episodes in fifty, which is indistinguishable from zero at that sample size. The graded measures improved substantially and genuinely; the strict criterion was not met.

The algorithm comparison is not a fair test. SAC received a sixteen-trial hyperparameter search; TD3, DDPG and PPO were run at library defaults. Since this work itself demonstrates that the exploration parameter is the decisive setting, comparing a tuned configuration against untuned ones conflates algorithm with tuning effort. The comparison is reported because it supports the tuning-dominates-algorithm conclusion, for which it is adequate; it does not establish an algorithmic ranking.

The comparison is entirely computational. No physical arm, no electromyography, no human participant. Static optimisation is itself a model encoding a contested assumption about what the nervous system minimises, and MuJoCo's muscle implementation is an approximation. Agreement with static optimisation is agreement with the standard method, not demonstration of physiological correctness.

The muscle set is incomplete. The rotator cuff is absent, which required constraining shoulder rotation (section 3.2). Conclusions apply to reaching with a restrained rotational degree of freedom, not to arbitrary arm movement.

8.9.3 On methodology

Six substantive defects were identified during this work, documented in chapter 5. Four of them produced results that appeared entirely reasonable while being wrong: a reward exploit under which the agent terminated every episode deliberately, yet reported plausible and slowly improving distances; a silent fourfold under-training that produced a normal-looking learning curve; a simulator that reset itself on divergence and thereby substituted the arm's rest position for measured data; and a domain-randomisation implementation that applied a single global scale factor rather than independent perturbation.

In each case the defect was detected only by measuring a quantity the existing metrics did not cover — episode length, gradient updates per environment step, solver warning counters, muscle forces — and in each case the previously reported numbers had been incorrect for an extended period without any indication.

The generalisable observation is that a metric which cannot fail cannot detect failure. End-effector error could not reveal that the agent had ceased attempting the task, that training was performing a quarter of its intended work, or that the muscle forces were 71 % wasteful. Each required a measurement designed specifically for the failure mode, which in turn required the failure mode to be hypothesised first. This is the reason chapter 5 documents the errors at the same length as the results.

8.10 Future directions

Isolate the hyperparameter responsible for the precision plateau. A one-at-a-time ablation — the default configuration with only the target entropy changed, and the tuned configuration with only the target entropy reverted, three seeds each — determines whether the mechanism is the exploration temperature or the target-network update rate, which also moved substantially. Approximately four hours. Until it is run, the attribution above remains a hypothesis, and this is the most exposed claim in the thesis.

Replicate the principal result across seeds. Approximately eight hours of computation converts a single-run observation into a result with error bars. This should precede any publication.

Train with the classical criterion as the effort term. Section 7.13 predicts

that a reward whose effort term is exactly $\sum_i (f_i/F_{\max,i})^2$ should close the 1.71× discrepancy. This distinguishes a limitation of reinforcement learning from a limitation of the reward chosen here — two conclusions with entirely different implications.

Tune the exploration parameter of every algorithm on an equal budget. Target entropy for SAC, action-noise scale for TD3 and DDPG, entropy coefficient for PPO. This is the only route to a fair four-way comparison, given that this work identifies exploration as the decisive axis.

Compare against electromyography. Static optimisation is a model; measured muscle activity is closer to ground truth. This step would convert a computational comparison into a physiological claim, and is the natural extension toward the clinical applications that motivate the work.

Repeat the synergy and co-contraction analyses on a valid policy. The implementation now exists; the earlier results were computed on policies since established to be invalid.

8.11 Closing remark

The contribution of this thesis is narrower than initially envisaged but rests on evidence that can be inspected. Every quantitative statement traces to a named artifact produced by a documented procedure, and where a result is weak, uncertain or absent, the text records it as such.

That a learned controller recovers the coordination structure of a classical optimisation criterion without ever being shown it is a genuine and encouraging result. That it does so while expending 71 % more effort than necessary is an equally genuine limitation, and identifying it required measuring muscle forces directly — which no amount of additional accuracy on the reaching task would ever have revealed.

Bibliographie

[1] Emanuel Todorov, Tom Erez, and Yuval Tassa. “MuJoCo: A Physics Engine for Model-Based Control”. In: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2012, pp. 5026–5033. DOI: 10.1109/IROS.2012.6386109.

[2] A. V. Hill. “The Heat of Shortening and the Dynamic Constants of Muscle”. In: *Proceedings of the Royal Society of London. Series B, Biological Sciences* 126.843 (1938), pp. 136–195. DOI: 10.1098/rspb.1938.0050.

[3] Felix E. Zajac. “Muscle and Tendon: Properties, Models, Scaling, and Application to Biomechanics and Motor Control”. In: *Critical Reviews in Biomedical Engineering* 17.4 (1989), pp. 359–411.

[4] Darryl G. Thelen. “Adjustment of Muscle Mechanics Model Parameters to Simulate Dynamic Contractions in Older Adults”. In: *Journal of Biomechanical Engineering* 125.1 (2003), pp. 70–77. DOI: 10.1115/1.1531112.

[5] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. “Stable-Baselines3: Reliable Reinforcement Learning Implementations”. In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8.

[6] Timothy P. Lillicrap et al. “Continuous Control with Deep Reinforcement Learning”. In: *International Conference on Learning Representations (ICLR)*. 2016.

[7] Scott Fujimoto, Herke van Hoof, and David Meger. “Addressing Function Approximation Error in Actor-Critic Methods”. In: *International Conference on Machine Learning (ICML)*. Vol. 80. 2018, pp. 1587–1596.

[8] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”. In: *International Conference on Machine Learning (ICML)*. Vol. 80. 2018, pp. 1861–1870.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).

[10] Gert Kwakkel, Boudewijn J. Kollen, Jeroen van der Grond, and Arie J. H. Prevo. "Probability of Regaining Dexterity in the Flaccid Upper Limb: Impact of Severity of Paresis and Time Since Onset in Acute Stroke". In: *Stroke* 34.9 (2003), pp. 2181–2186. DOI: 10.1161/01.STR.0000087172.16305.CD.

[11] Scott L. Delp et al. "OpenSim: Open-Source Software to Create and Analyze Dynamic Simulations of Movement". In: *IEEE Transactions on Biomedical Engineering* 54.11 (2007), pp. 1940–1950. DOI: 10.1109/TBME.2007.901024.

[12] Volodymyr Mnih et al. "Human-Level Control through Deep Reinforcement Learning". In: *Nature* 518.7540 (2015), pp. 529–533. DOI: 10.1038/nature14236.

[13] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press, 2018. ISBN: 9780262039246.

[14] Vittorio Caggiano, Huawei Wang, Guillaume Durandau, Massimo Sartori, and Vikash Kumar. "MyoSuite: A Contact-Rich Simulation Suite for Musculoskeletal Motor Control". In: *Proceedings of Machine Learning Research* 168 (2022). L4DC 2022, pp. 492–507.

[15] Olivier Codol, Jonathan A. Michaels, Mehrdad Kashefi, J. Andrew Pruszynski, and Paul L. Gribble. "MotorNet: A Python Toolbox for Controlling Differentiable Biomechanical Effectors with Artificial Neural Networks". In: *eLife* 12 (2024), RP88591. DOI: 10.7554/eLife.88591.

[16] Lukasz Kidzinski et al. "Learning to Run Challenge Solutions: Adapting Reinforcement Learning Methods for Neuromusculoskeletal Environments". In: *The NIPS '17 Competition: Building Intelligent Systems*. Springer, 2018, pp. 121–153. DOI: 10.1007/978-3-319-94042-7_7.

[17] World Health Organization. *Global Status Report on Rehabilitation*. https://www.who.int/health-topics/rehabilitation. Geneva, 2023.

[18] Valery L. Feigin et al. "World Stroke Organization (WSO): Global Stroke Fact Sheet 2022". In: *International Journal of Stroke* 17.1 (2022), pp. 18–29. DOI: 10.1177/17474930211065917.

[19] Saloua Mrabet, Nadia Ben Ali, Mariem Kchaou, and Samir Belal. "Épidémiologie des accidents vasculaires cérébraux en Tunisie". In: *Revue Neurologique* 171 (2015), A194. DOI: 10.1016/j.neurol.2015.01.436.

[20] Albert C. Lo et al. "Robot-Assisted Therapy for Long-Term Upper-Limb Impairment after Stroke". In: *New England Journal of Medicine* 362.19 (2010), pp. 1772–1783. DOI: 10.1056/NEJMoa0911341.

[21] Katherine R. S. Holzbaur, Wendy M. Murray, and Scott L. Delp. “A Model of the Upper Extremity for Simulating Musculoskeletal Surgery and Analyzing Neuromuscular Control”. In: *Annals of Biomedical Engineering* 33.6 (2005), pp. 829–840. DOI: 10.1007/s10439-005-3320-7.

[22] David A. Winter. *Biomechanics and Motor Control of Human Movement*. 4th ed. Hoboken, NJ: John Wiley & Sons, 2009. ISBN: 9780470398180.

[23] A. M. Gordon, A. F. Huxley, and F. J. Julian. “The Variation in Isometric Tension with Sarcomere Length in Vertebrate Muscle Fibres”. In: *The Journal of Physiology* 184.1 (1966), pp. 170–192. DOI: 10.1113/jphysiol.1966.sp007909.

[24] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. “Optuna: A Next-Generation Hyperparameter Optimization Framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019, pp. 2623–2631. DOI: 10.1145/3292500.3330701.

[25] K. Falconer and David A. Winter. “Quantitative Assessment of Co-Contraction at the Ankle Joint in Walking”. In: *Electromyography and Clinical Neurophysiology* 25.2–3 (1985), pp. 135–149.

[26] Lalit J. Bhargava, Marcus G. Pandy, and Frank C. Anderson. “A Phenomenological Model for Estimating Metabolic Energy Consumption in Muscle Contraction”. In: *Journal of Biomechanics* 37.1 (2004), pp. 81–88. DOI: 10.1016/S0021-9290(03)00239-2.

[27] Andrea d’Avella, Philippe Saltiel, and Emilio Bizzi. “Combinations of Muscle Synergies in the Construction of a Natural Motor Behavior”. In: *Nature Neuroscience* 6.3 (2003), pp. 300–308. DOI: 10.1038/nn1010.

[28] Emanuel Todorov. “Optimality Principles in Sensorimotor Control”. In: *Nature Neuroscience* 7.9 (2004), pp. 907–915. DOI: 10.1038/nn1309.

[29] Stephen H. Scott. “Optimal Feedback Control and the Neural Basis of Volitional Motor Control”. In: *Nature Reviews Neuroscience* 5.7 (2004), pp. 532–546. DOI: 10.1038/nrn1427.

[30] Tamar Flash and Neville Hogan. “The Coordination of Arm Movements: An Experimentally Confirmed Mathematical Model”. In: *The Journal of Neuroscience* 5.7 (1985), pp. 1688–1703. DOI: 10.1523/JNEUROSCI.05-07-01688.1985.

[31] Christopher M. Harris and Daniel M. Wolpert. “Signal-Dependent Noise Determines Motor Planning”. In: *Nature* 394.6695 (1998), pp. 780–784. DOI: 10.1038/29528.

[32] Emanuel Todorov and Michael I. Jordan. “Optimal Feedback Control as a Theory of Motor Coordination”. In: *Nature Neuroscience* 5.11 (2002), pp. 1226–1235. DOI: 10.1038/nn963.

[33] Daniel M. Wolpert and Mitsuo Kawato. “Multiple Paired Forward and Inverse Models for Motor Control”. In: *Neural Networks* 11.7–8 (1998), pp. 1317–1329. DOI: 10.1016/S0893-6080(98)00066-5.

[34] Lena H. Ting and J. Lucas McKay. “Neuromechanics of Muscle Synergies for Posture and Movement”. In: *Current Opinion in Neurobiology* 17.6 (2007), pp. 622–628. DOI: 10.1016/j.conb.2008.01.002.

[35] Emilio Bizzi and Vincent C. K. Cheung. “The Neural Origin of Muscle Synergies”. In: *Frontiers in Computational Neuroscience* 7 (2013), p. 51. DOI: 10.3389/fncom.2013.00051.

[36] Roy D. Crowninshield and Richard A. Brand. “A Physiologically Based Criterion of Muscle Force Prediction in Locomotion”. In: *Journal of Biomechanics* 14.11 (1981), pp. 793–801. DOI: 10.1016/0021-9290(81)90035-X.

[37] Frank C. Anderson and Marcus G. Pandy. “Static and Dynamic Optimization Solutions for Gait Are Practically Equivalent”. In: *Journal of Biomechanics* 34.2 (2001), pp. 153–161. DOI: 10.1016/S0021-9290(00)00155-X.

[38] Yuval Tassa et al. “dm_control: Software and Tasks for Continuous Control”. In: *Software Impacts* 6 (2020), p. 100022. DOI: 10.1016/j.simpa.2020.100022.

[39] Matthew Millard, Thomas Uchida, Ajay Seth, and Scott L. Delp. “Flexing Computational Muscle: Modeling and Simulation of Musculotendon Dynamics”. In: *Journal of Biomechanical Engineering* 135.2 (2013), p. 021005. DOI: 10.1115/1.4023390.

[40] Richard Bellman. *Dynamic Programming*. Princeton, NJ: Princeton University Press, 1957.

[41] Vijay R. Konda and John N. Tsitsiklis. “Actor-Critic Algorithms”. In: *Advances in Neural Information Processing Systems* 12 (2000), pp. 1008–1014.

[42] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *International Conference on Learning Representations (ICLR)*. 2016.

[43] Aravind Rajeswaran, Kendall Lowrey, Emanuel Todorov, and Sham Kakade. “Towards Generalization and Simplicity in Continuous Control”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017, pp. 6550–6561.

[44] Seungmoon Song et al. “Deep Reinforcement Learning for Modeling Human Locomotion Control in Neuromechanical Simulation”. In: *Journal of NeuroEngineering and Rehabilitation* 18.1 (2021), p. 126. DOI: 10.1186/s12984-021-00919-y.

[45] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. “DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills”. In: *ACM Transactions on Graphics* 37.4 (2018), 143:1–143:14. DOI: 10.1145/3197517.3201311.

[46] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World”. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133.

[47] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. “Sim-to-Real Transfer of Robotic Control with Dynamics Randomization”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 3803–3810. DOI: 10.1109/ICRA.2018.8460528.

[48] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. “Deep Reinforcement Learning That Matters”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018, pp. 3207–3214. DOI: 10.1609/aaai.v32i1.11694.

[49] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. “Deep Reinforcement Learning at the Edge of the Statistical Precipice”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 34. 2021, pp. 29304–29320.

[50] Wendy M. Murray, Scott L. Delp, and Thomas S. Buchanan. “Variation of Muscle Moment Arms with Elbow and Forearm Position”. In: *Journal of Biomechanics* 28.5 (1995), pp. 513–525. DOI: 10.1016/0021-9290(94)00114-J.

[51] Reza Shadmehr and John W. Krakauer. “A Computational Neuroanatomy for Motor Control”. In: *Experimental Brain Research* 185.3 (2008), pp. 359–381. DOI: 10.1007/s00221-008-1280-5.

[52] Xavier Glorot and Yoshua Bengio. “Understanding the Difficulty of Training Deep Feedforward Neural Networks”. In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Vol. 9. 2010, pp. 249–256.

[53] Joelle Pineau et al. “Improving Reproducibility in Machine Learning Research”. In: *Journal of Machine Learning Research* 22.164 (2021), pp. 1–20.

A MJCF File of the Musculoskeletal Model

Extrait du fichier MJCF décrivant le modèle du bras à neuf muscles. Les propriétés biomécaniques (longueurs optimales, forces isométriques maximales, points d'insertion) sont issues de [21] ; les paramètres inertiels proviennent de [22]. Le fichier complet est disponible sur le dépôt Git associé à ce mémoire.

```
<mujoco model="arm_9muscles">
  <option timestep="0.002" iterations="50" solver="Newton"
          integrator="implicit" cone="elliptic"/>

  <default>
    <muscle ctrllimited="true" ctrlrange="0 1"
            force="-1" scale="200" lmin="0.5" lmax="1.6"
            vmax="10" fpmax="1.5" fvmax="1.4"/>
  </default>

  <worldbody>
    <body name="humerus" pos="0 0 -0.3">
      <joint name="shoulder_flex" type="hinge" axis="0 1 0" range="
         -0.5 2.5" damping="0.1"/>
      <joint name="shoulder_abd"  type="hinge" axis="1 0 0" range="
         -0.5 2.5" damping="0.1"/>
      <joint name="shoulder_rot"  type="hinge" axis="0 0 1" range="
         -1.5 1.5" damping="0.1"/>
      <geom type="capsule" fromto="0 0 0 0 0 -0.3" size="0.04" mass
         ="1.98"/>

      <!-- Muscle origin sites (humerus) -->
      <site name="bic_origin_long" pos="0     0.01  0.05"/>
      <site name="bic_s_origin"    pos="0.01  0.01  0.03"/>
      <site name="bra_origin"      pos="0     0.00 -0.15"/>
      <site name="tri_l_origin"    pos="0    -0.01  0.02"/>
      <site name="delt_a_origin"   pos="0.05  0.00  0.05"/>
      <site name="delt_m_origin"   pos="0.00  0.05  0.05"/>
      <site name="delt_p_origin"   pos="-0.05 0.00  0.05"/>

      <body name="radius" pos="0 0 -0.3">
```

```
28        <joint name="elbow_flex" type="hinge" axis="0 1 0" range="0
              2.7" damping="0.1"/>
29        <geom type="capsule" fromto="0 0 0 0 0 -0.27" size="0.03"
             mass="1.18"/>
30
31        <!-- Via-point and insertion sites (forearm) -->
32        <site name="bic_via_1"        pos="0     0.02 -0.02"/>
33        <site name="bic_insertion"    pos="0     0.02 -0.08"/>
34        <site name="bic_s_insertion"  pos="0     0.02 -0.08"/>
35        <site name="bra_insertion"    pos="0     0.01 -0.05"/>
36        <site name="tri_l_insertion"  pos="0    -0.01 -0.01"/>
37        <site name="tri_la_origin"    pos="0.01 -0.01 -0.10"/>
38        <site name="tri_la_insertion" pos="0    -0.01 -0.01"/>
39        <site name="tri_m_origin"     pos="-0.01 -0.01 -0.10"/>
40        <site name="tri_m_insertion"  pos="0    -0.01 -0.01"/>
41        <site name="delt_a_insertion" pos="0.03  0.00 -0.15"/>
42        <site name="delt_m_insertion" pos="0.00  0.03 -0.15"/>
43        <site name="delt_p_insertion" pos="-0.03 0.00 -0.15"/>
44
45        <body name="hand" pos="0 0 -0.27">
46          <geom type="sphere" size="0.04" mass="0.45"/>
47          <site name="end_effector" pos="0 0 0"/>
48        </body>
49      </body>
50    </body>
51  </worldbody>
52
53  <tendon>
54    <spatial name="biceps_long"  width="0.003">
55      <site site="bic_origin_long"/>
56      <site site="bic_via_1"/>
57      <site site="bic_insertion"/>
58    </spatial>
59    <spatial name="biceps_short" width="0.003">
60      <site site="bic_s_origin"/>
61      <site site="bic_s_insertion"/>
62    </spatial>
63    <spatial name="brachialis"   width="0.003">
64      <site site="bra_origin"/>
65      <site site="bra_insertion"/>
66    </spatial>
67    <spatial name="triceps_long" width="0.003">
68      <site site="tri_l_origin"/>
69      <site site="tri_l_insertion"/>
70    </spatial>
```

```
    <spatial name="triceps_lat"  width="0.003">
      <site site="tri_la_origin"/>
      <site site="tri_la_insertion"/>
    </spatial>
    <spatial name="triceps_med"  width="0.003">
      <site site="tri_m_origin"/>
      <site site="tri_m_insertion"/>
    </spatial>
    <spatial name="deltoid_ant"  width="0.003">
      <site site="delt_a_origin"/>
      <site site="delt_a_insertion"/>
    </spatial>
    <spatial name="deltoid_med"  width="0.003">
      <site site="delt_m_origin"/>
      <site site="delt_m_insertion"/>
    </spatial>
    <spatial name="deltoid_post" width="0.003">
      <site site="delt_p_origin"/>
      <site site="delt_p_insertion"/>
    </spatial>
  </tendon>

  <actuator>
    <muscle name="biceps_long"  tendon="biceps_long"  force="624"
       lengthrange="0.01 0.50"/>
    <muscle name="biceps_short" tendon="biceps_short" force="435"
       lengthrange="0.01 0.50"/>
    <muscle name="brachialis"   tendon="brachialis"   force="987"
       lengthrange="0.01 0.50"/>
    <muscle name="triceps_long" tendon="triceps_long" force="798"
       lengthrange="0.01 0.50"/>
    <muscle name="triceps_lat"  tendon="triceps_lat"  force="624"
       lengthrange="0.01 0.50"/>
    <muscle name="triceps_med"  tendon="triceps_med"  force="624"
       lengthrange="0.01 0.50"/>
    <muscle name="deltoid_ant"  tendon="deltoid_ant"  force="1142"
       lengthrange="0.01 0.50"/>
    <muscle name="deltoid_med"  tendon="deltoid_med"  force="1142"
       lengthrange="0.01 0.50"/>
    <muscle name="deltoid_post" tendon="deltoid_post" force="259"
       lengthrange="0.01 0.50"/>
  </actuator>

  <sensor>
    <actuatorfrc name="f_bl"  actuator="biceps_long"/>
```

```
107     <actuatorfrc name="f_bs"  actuator="biceps_short"/>
108     <actuatorfrc name="f_bra" actuator="brachialis"/>
109     <actuatorfrc name="f_tl"  actuator="triceps_long"/>
110     <actuatorfrc name="f_tla" actuator="triceps_lat"/>
111     <actuatorfrc name="f_tm"  actuator="triceps_med"/>
112     <actuatorfrc name="f_da"  actuator="deltoid_ant"/>
113     <actuatorfrc name="f_dm"  actuator="deltoid_med"/>
114     <actuatorfrc name="f_dp"  actuator="deltoid_post"/>
115     <jointpos name="shf_q" joint="shoulder_flex"/>
116     <jointvel name="shf_v" joint="shoulder_flex"/>
117   </sensor>
118 </mujoco>
```

Listing A.1. Extrait du fichier MJCF (arm.xml).

B Gymnasium Environment (Python)

Implémentation Python de l'environnement `ArmMusculoEnv` héritant de `gymnasium.Env`, intégrant la randomisation de domaine et le calcul de la récompense définie en (3.4).

```python
import numpy as np
import gymnasium as gym
from gymnasium import spaces
import mujoco
import mujoco.viewer


class ArmMusculoEnv(gym.Env):
    """Environnement musculo-squelettique pour tache d'atteinte 3D."""

    metadata = {"render_modes": ["human", "rgb_array"]}

    def __init__(self, model_path="arm.xml", domain_rand=True,
                 max_steps=1000, render_mode=None, seed=None):
        super().__init__()
        self.model = mujoco.MjModel.from_xml_path(model_path)
        self.data = mujoco.MjData(self.model)
        self.domain_rand = domain_rand
        self.max_steps = max_steps
        self.render_mode = render_mode
        self.rng = np.random.default_rng(seed)

        # 9 muscles, activation dans [0,1]
        self.action_space = spaces.Box(
            low=0.0, high=1.0, shape=(9,), dtype=np.float32)

        # 38 dimensions : 4q, 4qdot, 9 lengths, 9 forces, 9 acts, 3 err
        self.observation_space = spaces.Box(
            low=-np.inf, high=np.inf, shape=(38,), dtype=np.float32)

        # Poids de recompense optimises par Optuna
```

```
        self.w1, self.w2, self.w3 = 1.0, 0.005, 0.01
        self.w4, self.w5 = 0.1, 0.5

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        mujoco.mj_resetData(self.model, self.data)

        # Randomisation de domaine
        if self.domain_rand:
            # Masses +/- 15%
            self.model.body_mass[:] *= self.rng.uniform(0.85, 1.15)
            # Parametres de Hill Fmax +/- 20%
            for i in range(self.model.nu):
                self.model.actuator_gainprm[i, 2] *= \
                    self.rng.uniform(0.80, 1.20)
            # Amortissement articulaire
            self.model.dof_damping[:] *= self.rng.uniform(0.70,
                1.30)

        # Cible aleatoire dans l'espace atteignable
        self.target = self.rng.uniform(
            low=[-0.20, -0.30, -0.45], high=[0.45, 0.30, 0.00])

        self.steps = 0
        self.prev_action = np.zeros(9, dtype=np.float32)
        obs = self._get_obs()
        return obs, {}

    def step(self, action):
        action = np.clip(action, 0.0, 1.0).astype(np.float32)
        self.data.ctrl[:] = action
        # Bruit moteur (OU) pour robustesse
        if self.domain_rand:
            self.data.ctrl[:] += self.rng.normal(0, 0.02, size=9)
            self.data.ctrl[:] = np.clip(self.data.ctrl, 0, 1)
        mujoco.mj_step(self.model, self.data)

        obs = self._get_obs()
        err = np.linalg.norm(obs[-3:])
        e_metab = np.sum(self.data.actuator_force ** 2)
        d_act = np.sum((action - self.prev_action) ** 2)
        v = np.linalg.norm(self.data.qvel[:4])

        reward = -(self.w1 * err**2 + self.w2 * e_metab +
                   self.w3 * d_act) + self.w4
```

```
76          success = (err < 0.02) and (v < 0.1)
77          if success:
78              reward += self.w5
79
80          self.steps += 1
81          terminated = bool(success) or (err > 0.60) or \
82                       np.any(np.abs(self.data.qvel[:4]) > 50)
83          truncated = self.steps >= self.max_steps
84
85          self.prev_action = action
86          info = {"err": err, "success": success, "energy": e_metab}
87          return obs, float(reward), terminated, truncated, info
88
89      def _get_obs(self):
90          q = self.data.qpos[:4]
91          qdot = self.data.qvel[:4]
92          lm = np.array([self.data.actuator_length[i]
93                         for i in range(9)])
94          fm = np.array([self.data.actuator_force[i]
95                         for i in range(9)])
96          act = np.array([self.data.act[i] for i in range(9)])
97          ee = self.data.site_xpos[
98              mujoco.mj_name2id(
99                  self.model, mujoco.mjtObj.mjOBJ_SITE, "end_effector
                        ")]
100         err = self.target - ee
101         return np.concatenate([q, qdot, lm, fm, act, err]).astype(
102             np.float32)
```

Listing B.1. Extrait de `arm_env.py`.

C SAC Training Script

Training script using Stable-Baselines3 for SAC, with observation normalization, periodic evaluation, and saving of the best-performing models.

```
import argparse, os, numpy as np
from stable_baselines3 import SAC
from stable_baselines3.common.vec_env import (
    SubprocVecEnv, VecNormalize)
from stable_baselines3.common.callbacks import EvalCallback
from arm_env import ArmMusculoEnv

def make_env(seed, domain_rand=True):
    def _init():
        env = ArmMusculoEnv(domain_rand=domain_rand, seed=seed)
        env.reset(seed=seed)
        return env
    return _init

def main(args):
    os.makedirs(args.logdir, exist_ok=True)

    # Environnements vectorises
    train_envs = SubprocVecEnv(
        [make_env(args.seed + i) for i in range(args.n_envs)])
    train_envs = VecNormalize(
        train_envs, norm_obs=True, norm_reward=True, clip_obs=10.0)

    eval_env = SubprocVecEnv([make_env(args.seed + 999)])
    eval_env = VecNormalize(
        eval_env, norm_obs=True, norm_reward=False,
        training=False)

    # Hyperparametres issus d'Optuna (Annexe D)
    model = SAC(
        "MlpPolicy", train_envs,
        learning_rate=3e-4,
        buffer_size=1_000_000,
        batch_size=256,
        tau=0.005,
        gamma=0.99,
        train_freq=1,
```

```
        gradient_steps=1,
        ent_coef="auto",
        target_entropy="auto",
        policy_kwargs=dict(net_arch=[256, 256]),
        seed=args.seed,
        verbose=1,
        tensorboard_log=args.logdir)

    callback = EvalCallback(
        eval_env,
        best_model_save_path=args.logdir,
        log_path=args.logdir,
        eval_freq=5000,
        n_eval_episodes=20,
        deterministic=True)

    model.learn(total_timesteps=args.steps, callback=callback)
    model.save(os.path.join(args.logdir, "final_model"))
    train_envs.save(os.path.join(args.logdir, "vecnormalize.pkl"))

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--seed", type=int, required=True)
    parser.add_argument("--steps", type=int, default=2_000_000)
    parser.add_argument("--n_envs", type=int, default=4)
    parser.add_argument("--logdir", type=str, default="runs/sac")
    args = parser.parse_args()
    main(args)
```

Listing C.1. Excerpt from `train_sac.py`.

D Hill Integration by Newton–Raphson

Numerical solution of the muscle equilibrium equation (2.9) by Newton–Raphson iterations, used at each simulation step to determine the fiber length ℓ_M from the musculotendon length ℓ_{MT} and the activation level a.

```
import numpy as np

def solve_muscle_equilibrium(L_MT, a, L_M0, L_S0, alpha0,
                             F_max, epsilon=1e-6, max_iter=50):
    """
    Resout f_CE + f_PEE - f_SEE = 0 pour L_M.
    Retourne (L_M, L_S, F_muscle) par iterations de Newton-Raphson.
    """
    # Initialisation : hypothese L_S = L_S0 (tendon a longueur
       nominale)
    L_M = L_MT - L_S0
    L_M = max(0.5 * L_M0, min(1.6 * L_M0, L_M))  # bornes

    for it in range(max_iter):
        # Angle de pennation
        sin_alpha = (L_M0 * np.sin(alpha0)) / L_M
        cos_alpha = np.sqrt(max(1.0 - sin_alpha**2, 1e-6))

        # Longueur du tendon
        L_S = L_MT - L_M * cos_alpha

        # Force active (gaussienne centree sur L_M0)
        f_l = np.exp(-((L_M / L_M0 - 1.0) / 0.45) ** 2)
        f_CE = a * f_l * F_max

        # Force passive (exponentielle)
        if L_M > L_M0:
            f_PEE = F_max * (np.exp(5.0 * (L_M / L_M0 - 1.0)) -
               1.0) \
                    / (np.exp(5.0) - 1.0)
        else:
            f_PEE = 0.0

```

```
        # Force tendineuse (quadratique en deformation)
        eps_s = (L_S - L_S0) / L_S0
        if eps_s > 0:
            f_SEE = F_max * 37.5 * eps_s ** 2
        else:
            f_SEE = 0.0

        # Residu d'equilibre
        res = (f_CE + f_PEE) * cos_alpha - f_SEE
        if abs(res) < epsilon:
            break

        # Derivee numerique pour mise a jour
        dL = 1e-5 * L_M0
        L_M_p = L_M + dL
        # (recalcul rapide f_CE, f_PEE, f_SEE a L_M_p omis pour
            concision)
        # Puis : L_M -= res / (dRes / dL_M)
        L_M -= 0.5 * res / (F_max / L_M0)  # simplification pas
            adaptatif
        L_M = max(0.5 * L_M0, min(1.6 * L_M0, L_M))

    return L_M, L_S, (f_CE + f_PEE) * cos_alpha
```

Listing D.1. Musculotendon equilibrium solver.

E Detailed Statistical Protocol

This protocol was not applied to any result in this thesis, and no p-value, effect size or confidence interval appears anywhere in the text. It is retained here because the implementation is complete and correct, and because the seed count — not the analysis — is what prevents its use.

With the three seeds the available hardware permitted (section 5.3.2), the two-sided Wilcoxon signed-rank test has a minimum attainable p-value of 0.25. Running it would produce numbers that cannot reach significance under any threshold while giving the appearance of inferential rigour. An earlier version of this work reported p-values from this protocol over a purported ten seeds; those analyses were never executed and are withdrawn (section 8.6).

The code below therefore documents what *should* be run once the replication recommended in section 8.10 is complete: Wilcoxon signed-rank test for paired samples, Bonferroni correction for multiple comparisons, BCa bootstrap confidence intervals, and rank-biserial effect size.

```
import numpy as np
from scipy import stats
from scipy.stats import wilcoxon

def rank_biserial(x, y):
    """Taille d'effet rank-biserial pour deux echantillons apparies."""
    diff = np.asarray(x) - np.asarray(y)
    pos = np.sum(diff > 0)
    neg = np.sum(diff < 0)
    return (pos - neg) / (pos + neg) if (pos + neg) > 0 else 0.0

def bootstrap_bca(data, stat_fn=np.mean, n_boot=10000,
                  alpha=0.05, rng=None):
    """Intervalle de confiance BCa (biais corrige, accelere)."""
    rng = np.random.default_rng(rng)
    data = np.asarray(data)
    n = len(data)
    boot_stats = np.array([
        stat_fn(rng.choice(data, size=n, replace=True))
        for _ in range(n_boot)])
    theta_hat = stat_fn(data)
```

```
22
23      # Correction de biais
24      z0 = stats.norm.ppf(np.mean(boot_stats < theta_hat))
25      # Acceleration via jackknife
26      jack = np.array([
27          stat_fn(np.delete(data, i)) for i in range(n)])
28      jack_mean = np.mean(jack)
29      num = np.sum((jack_mean - jack) ** 3)
30      den = 6.0 * (np.sum((jack_mean - jack) ** 2) ** 1.5 + 1e-12)
31      a = num / den
32
33      z_a = stats.norm.ppf(alpha / 2)
34      z_1a = stats.norm.ppf(1 - alpha / 2)
35      a1 = stats.norm.cdf(z0 + (z0 + z_a) / (1 - a * (z0 + z_a)))
36      a2 = stats.norm.cdf(z0 + (z0 + z_1a) / (1 - a * (z0 + z_1a)))
37      return (np.quantile(boot_stats, a1),
38              np.quantile(boot_stats, a2))
39
40  def compare_algorithms(results):
41      """results: dict {algo: array[10] de taux de succes}"""
42      keys = list(results.keys())
43      n_tests = len(keys) * (len(keys) - 1) // 2
44      alpha_corrected = 0.05 / n_tests  # Bonferroni
45
46      for i, a in enumerate(keys):
47          for b in keys[i+1:]:
48              stat, p = wilcoxon(results[a], results[b],
49                                 alternative='two-sided')
50              rb = rank_biserial(results[a], results[b])
51              ci_a = bootstrap_bca(results[a])
52              print(f"{a} vs {b}: W={stat}, p={p:.4f}, "
53                    f"rb={rb:.2f}, signif={p < alpha_corrected}")
```

Listing E.1. Inter-algorithm statistical analysis.

« Nous n'imitons pas la nature pour la copier, mais pour comprendre les principes qui en ordonnent la complexité — et mettre ces principes au service de ceux qui peinent à se mouvoir. »

— *Toward a biologically inspired neuroprosthetics*